Attribution-Patching-Guided Replay Refinement for Task-Vector Merging

Anonymous Authors
Draft proposal

August 1, 2026

Abstract

We study the task of merging fine-tuned variants of a shared pretrained model when a small labeled replay buffer for each task is available. After an initial task-arithmetic merge, we refine the merged model with task gradients while using the known task experts as coordinate-wise trust anchors. The resulting update admits two equivalent motivations that we develop in parallel. As a parameter-space analogue of attribution patching, the effect of moving the current model toward a task expert is approximated by a first-order loss attribution, and only coordinates for which the expert direction is locally loss-decreasing are used. As an expert-anchored trust-region replay refinement, the departure from ordinary replay gradient descent is that each coordinate's step is weighted by its distance to the expert, gated to keep only steps that move toward the expert, and clipped so it cannot overshoot—the same coordinates that the attribution view selects. Both perspectives lead to a single refinement algorithm.

1 Related Work and Background

Model merging and weight-space averaging. Model merging combines multiple fine-tuned models into a single model in weight space, avoiding the inference-time overhead of ensembling. Model soups average fine-tuned checkpoints that lie in a compatible basin (Wortsman et al., 2022); Fisher merging weights parameters by Fisher information, motivated by a Laplace approximation to each model's local posterior (Matena and Raffel, 2022). These approaches show that homologous fine-tuned models can often be combined directly in parameter space, but they do not by themselves resolve interference between models specialized to different tasks.

Task vectors. Task arithmetic represents the effect of fine-tuning as a task vector

$$\tau_t = \theta_t - \theta_0, \tag{1}$$

where θ_0 is a shared pretrained model and θ_t is the model fine-tuned on task t. A merged model is then constructed as

$$\theta_{\text{merge}} = \theta_0 + \sum_{t \in \mathcal{T}} \lambda_t \tau_t, \tag{2}$$

where λ_t are scalar or tensor-wise coefficients. Task Arithmetic demonstrated that such vectors can be added or negated to steer model behavior (Ilharco et al., 2022). This formulation is attractive because it requires only checkpoints, but naive addition can over-count redundant directions and introduce sign or magnitude conflicts across tasks.

Interference-aware task-vector merging. A major line of work addresses interference between task vectors. TIES-Merging trims small-magnitude updates, resolves sign disagreement, and merges only sign-consistent components (Yadav et al., 2023); DARE randomly drops most delta parameters and rescales the rest (Yu et al., 2023), and DELLA generalizes this with magnitude-based sampling (Deep et al., 2024); Model Breadcrumbs removes both negligible and outlier perturbations (Davari and Belilovsky, 2023); Localize-and-Stitch localizes sparse task-relevant regions and stitches only those back into the pretrained model (He et al., 2024). All share the view that not all task-vector coordinates should be merged equally. The present proposal does not introduce another pruning rule; it asks whether a small amount of supervised replay can repair an initial merge while staying close to the known experts.

Data-dependent merging baselines. Several merging methods use small amounts of data or model outputs to estimate better combinations. Activated Parameter Locating (APL) estimates task-vector importance through a data-dependent contrast and then uses the result to guide pruning (Kong et al., 2024). RegMean formulates merging as a regression problem that minimizes prediction differences between the merged model and the candidate models (Jin et al., 2023). AdaMerging learns task-wise or layer-wise merge coefficients by minimizing an unsupervised entropy objective on unlabeled samples (Yang et al., 2024). These methods are important baselines because the proposed method also uses data, although it uses labeled replay buffers only for a few post-merge refinement steps rather than for constructing a pre-merge mask or a closed-form fusion rule.

Attribution patching. Attribution patching approximates the effect of an intervention by a first-order Taylor expansion, replacing many explicit interventions by gradient-based scores (Syed et al., 2024; Kramar et al., 2024). In parameter space the analogous approximation is a directional derivative of the task loss along a chosen displacement—here, the movement from the current merged model toward task i's expert. This gives a direct bridge to replay-buffer refinement: the attribution score decides which coordinates of the expert direction are locally predicted to decrease the task loss.

Shared and task-specific structure. Several recent methods recognize that task vectors contain both shared and exclusive information. Twin-Merging separates shared and exclusive task knowledge and dynamically combines them at inference time (Lu et al., 2024). Task Vector Bases clusters similar task vectors to compress and reuse shared directions, providing a theoretical and practical framework for reducing redundancy across related tasks (Zeng et al., 2025). FusionBench documents the need for standardized evaluation across model-fusion methods and includes GLUE-style language tasks among its benchmark families (Tang et al., 2024). Our proposal is complementary: rather than interpreting small attribution values as evidence that a coordinate can be pruned, we use the sign of a merge-to-expert attribution only as a local gate for refinement.

Benchmark settings. The primary diagnostic benchmark is GLUE-8 (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, excluding WNLI as is common) (Wang et al., 2018), with RoBERTa-base as the encoder backbone (Liu et al., 2019): the shared encoder is merged and evaluated with each task's own trained head. This multi-head setup is convenient and common, but it assumes oracle task identity at evaluation time and should not be the only evidence for the method. The recent merging literature evaluates on three benchmark families—CLIP/ViT vision task-vector suites, GLUE and text-to-text NLP, and decoder-only LLM merging, the last standardized by MergeBench with released full-parameter Llama/Gemma experts (Radford et al.,

2021; Ilharco et al., 2022; Tang et al., 2024; Wang et al., 2019; Raffel et al., 2020; Yu et al., 2023; He et al., 2025)—and we position the proposal against all three (protocol details in Section 4).

2 Methodology

2.1 Problem Setup

Let $\mathcal{T} = \{1, \ldots, T\}$ denote a set of tasks. All task experts are initialized from the same pretrained encoder $\theta_0 \in \mathbb{R}^d$. Fine-tuning on task t gives expert encoder parameters θ_t and task vector

$$\tau_t = \theta_t - \theta_0. \tag{3}$$

The mergeable encoder parameters are treated as a vector in $\mathbb{R}^d$. We write $r \in \{1, \ldots, d\}$ for a parameter coordinate. Task-specific heads are not merged in the primary GLUE experiments; the merged encoder is evaluated with the corresponding head for each task.

The initial merged encoder is

$$\theta^{(0)} = \theta_0 + \sum_{i=1}^{T} \lambda_i \tau_i, \tag{4}$$

where λ_i are fixed task-arithmetic coefficients, typically uniform or chosen by a small grid search. Let $\mathcal{D}_t^{\text{probe}}$ be a small replay buffer used for refinement, and let $\mathcal{D}_t^{\text{eval}}$ be the evaluation split used only for reporting performance. For each task i, let

$$\mathcal{L}_i(\theta; \mathcal{D}_i^{\text{probe}}) \tag{5}$$

denote the replay-buffer loss obtained by using encoder parameters θ with task i's prediction head. The proposed method does not use a base-relative saliency term and does not construct a pre-merge pruning mask. Its only data-dependent operation is a short replay-buffer refinement of the initial merge.

2.2 Trust-Region Replay Refinement

Starting from the task-arithmetic initialization in Equation (4), the natural data-dependent baseline is ordinary replay-buffer gradient descent, which takes an unconstrained step along the task gradients:

$$\theta^{(s+1)} = \theta^{(s)} - \eta \sum_{i=1}^{T} \nabla_\theta \mathcal{L}_i(\theta^{(s)}; \mathcal{D}_i^{\text{probe}}). \tag{6}$$

This baseline treats the merge like any other initialization and lets the replay gradients move each coordinate freely, with nothing tying the refined model to the known task experts. The central departure of our method is precisely this anchoring: rather than taking a free gradient step, every coordinate of the update is tied to the corresponding task expert. For each task i and coordinate r we (i) weight the negative-gradient step by the current distance $|v_{i,r}|$ to the expert, so that coordinates already close to the expert barely move while coordinates far from it move more; (ii) keep the step only where it moves the model *toward* the expert, discarding coordinates whose replay gradient would pull away from it; and (iii) clip the resulting step to the expert-distance trust region so that a single update can never overshoot the expert. These three operations—expert-distance weighting, the expert-agreement gate, and the trust-region clip—are what distinguish the refinement from ordinary replay gradient descent, and together they keep the whole refinement anchored to the known experts rather than letting it drift.

A secondary and comparatively minor design choice concerns how the per-task updates are scheduled within a refinement step. An aggregated expert-guided update would compute all u_i at $\theta^{(s)}$ and then apply $\eta \sum_i u_i$ at once; we instead apply each task's update immediately after computing it, so that its gradient, expert direction, and gate are evaluated at the same model state to which it is applied. Let $\theta^{(s,0)} = \theta^{(s)}$, and let π_s be the task order for sweep s; π_s can be fixed, cyclically rotated, or randomly permuted. For within-sweep index $k \in \{1, \ldots, T\}$, set $i = \pi_s(k)$ and compute

$$g_i^{(s,k)} = \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}}), \tag{7}$$

$$v_i^{(s,k)} = \theta_i - \theta^{(s,k-1)}, \tag{8}$$

$$m_{i,r}^{(s,k)} = \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}, \tag{9}$$

$$\widetilde{u}_{i,r}^{(s,k)} = -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}. \tag{10}$$

From this optimization side the coordinate mask $m_{i,r}^{(s,k)}$ arises as a trust-region acceptance rule rather than as an attribution score: we keep only those coordinates whose replay-gradient step also moves the current model toward expert i, and discard the coordinates where the replay step would pull away from the expert and out of the expert-anchored trust region. This is the same condition $g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0$ that Section 2.3 obtained from the attribution-patching sign in Equation (19), now read as *stay within the expert-anchored trust region* instead of *move along a locally loss-decreasing intervention*. The two perspectives therefore agree coordinate by coordinate. The raw update is first scaled by the learning rate η and *then* clipped by the the current distance to the expert, so that the applied per-coordinate step is bounded by $|v_{i,r}^{(s,k)}|$ regardless of η,

$$u_{i,r}^{(s,k)} = \text{clip}\Big(\eta\, \widetilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|\Big), \tag{11}$$

and the current model is immediately updated with this clipped step:

$$\theta^{(s,k)} = \theta^{(s,k-1)} + u_i^{(s,k)}. \tag{12}$$

After all tasks in the sweep have been processed, we set

$$\theta^{(s+1)} = \theta^{(s,T)}. \tag{13}$$

The robustness of the refinement comes primarily from the expert anchoring rather than from this scheduling: the distance weighting and gate keep every coordinate pointed at its expert, and the trust-region clip caps how far it can travel. The immediate application of each update, though secondary to the anchoring, has two motivations of its own. First, each update is applied at the same local model state on which its gradient, expert direction, and gate were computed. Second, the trust-region clipping in Equation (11) controls each single-task movement before the next task sees the modified model, rather than allowing several task vectors to accumulate into one larger coordinate step. Because the clip is applied *after* the learning-rate scaling, the per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η; for ≤ 1 a single update can never overshoot the expert in a coordinate, so the refinement stays within the expert-distance trust region and cannot diverge no matter how large η is. The learning rate then controls only how many coordinates are driven to the trust-region boundary, not the maximum per-coordinate step size. The sequential rule still does not guarantee Pareto improvement across tasks, because an update that helps task i can hurt task j. We therefore treat task ordering, cross-task interference, and worst-task retention as explicit

diagnostics rather than as consequences of the gate alone. In practice, one can also normalize $u_i^{(s,k)}$ by tensor-wise RMS before applying the single-task update; the defining operation remains the coordinate gate in Equation (19), whether it is read as an attribution-patching sign or as a trust-region acceptance rule. Under either reading the procedure is the single algorithm summarized in Algorithm 1.

Algorithm 1 Sequential expert-direction-gated replay refinement

Require: Pretrained encoder θ_0; task experts $\{\theta_i\}_{i=1}^T$; task heads; replay buffers $\{\mathcal{D}_i^{\text{probe}}\}_{i=1}^T$; task-arithmetic weights $\{\lambda_i\}_{i=1}^T$; steps S; learning rate η; gate threshold ϵ_{gate}.

$\tau_i \leftarrow \theta_i - \theta_0$ for all tasks i.
$\theta^{(0)} \leftarrow \theta_0 + \sum_{i=1}^T \lambda_i \tau_i$. ▷ or any other merge point (TIES, RegMean, ...)
for $s = 0, \ldots, S-1$ **do**
  $\theta^{(s,0)} \leftarrow \theta^{(s)}$.
  Choose a task order π_s over $\{1, \ldots, T\}$.
  for $k = 1, \ldots, T$ **do**
    $i \leftarrow \pi_s(k)$.
    $g_i^{(s,k)} \leftarrow \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}})$ using task i's head.
    $v_i^{(s,k)} \leftarrow \theta_i - \theta^{(s,k-1)}$.
    for each mergeable coordinate r **do**
      $m_{i,r}^{(s,k)} \leftarrow \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}$.
      $\widetilde{u}_{i,r}^{(s,k)} \leftarrow -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}$.
      $u_{i,r}^{(s,k)} \leftarrow \text{clip}(\eta\, \widetilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|)$. ▷ clip after learning-rate scaling
    end for
    $\theta^{(s,k)} \leftarrow \theta^{(s,k-1)} + u_i^{(s,k)}$.
  end for
  $\theta^{(s+1)} \leftarrow \theta^{(s,T)}$.
end for
return $\theta^{(S)}$.

The informal claim that the refinement "stays within the expert-anchored trust region and cannot diverge" can be made precise. Every gated, clipped step moves each coordinate between its current value and the corresponding expert coordinate, and such coordinatewise movement toward a point of the expert hull can never increase the distance to that hull. The following corollary states the resulting guarantee for Algorithm 1; it is an instance of a general geometric result, Theorem 1 in Appendix A, where we also show that the axis-aligned expert box is exactly the ℓ^1-geodesic convex hull of the experts (Proposition 1), so the box is not an arbitrary safe region but the natural coordinatewise notion of the experts' convex hull.

Corollary 1 (Box-distance monotonicity of the sequential refinement)**.** *Let*

$$B_{\text{exp}} := \prod_{r=1}^{d} [\underline{\theta}_r, \overline{\theta}_r], \qquad \underline{\theta}_r := \min_{j \in \mathcal{T}} \theta_{j,r}, \qquad \overline{\theta}_r := \max_{j \in \mathcal{T}} \theta_{j,r},$$

be the axis-aligned box hull of the task experts. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every $1 \le p \le \infty$,

$$d_p\Big(\theta^{(s,k)}, B_{\text{exp}}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\text{exp}}\Big).$$

Consequently, after any number S of sweeps,

$$d_p\Big(\theta^{(S)}, B_{\text{exp}}\Big) \le d_p\Big(\theta^{(0)}, B_{\text{exp}}\Big) \qquad \textit{for every } 1 \le p \le \infty.$$

Proof. Fix a sweep s, a within-sweep index k, and let $i = \pi_s(k)$. We first verify the hypothesis of Theorem 1 with

$$y = \theta^{(s,k-1)}, \qquad z = \theta^{(s,k)}, \qquad x_i = \theta_i.$$

It is enough to check that, for every coordinate r, the new coordinate $\theta_r^{(s,k)}$ lies between the previous coordinate $\theta_r^{(s,k-1)}$ and the expert coordinate $\theta_{i,r}$.

Write $v_r = v_{i,r}^{(s,k)} = \theta_{i,r} - \theta_r^{(s,k-1)}$. If the gate is off, then $m_{i,r}^{(s,k)} = 0$, hence $u_{i,r}^{(s,k)} = 0$, and the coordinate is unchanged. If the gate is on, then

$$g_{i,r}^{(s,k)} v_r < -\epsilon_{\text{gate}} \le 0,$$

so $-g_{i,r}^{(s,k)}$ has the same sign as v_r. Therefore the raw update $\widetilde{u}_{i,r}^{(s,k)} = -g_{i,r}^{(s,k)} |v_r| m_{i,r}^{(s,k)}$ points in the same direction as v_r. Multiplication by $\eta \ge 0$ preserves this direction, possibly making the update zero, and the symmetric clip in Equation (11) gives an applied update with the same weak sign as v_r and magnitude at most $|v_r|$. Thus there is an $\alpha_{i,r}^{(s,k)} \in [0,1]$ such that

$$u_{i,r}^{(s,k)} = \alpha_{i,r}^{(s,k)} v_{i,r}^{(s,k)}.$$

The same representation is assumed directly in the optional post-processing case. Hence

$$\theta_r^{(s,k)} = \theta_r^{(s,k-1)} + \alpha_{i,r}^{(s,k)} (\theta_{i,r} - \theta_r^{(s,k-1)}),$$

so $\theta_r^{(s,k)}$ lies between $\theta_r^{(s,k-1)}$ and $\theta_{i,r}$.

The box B_{exp} is exactly the axis-aligned hull of the expert points $\theta_1, \dots, \theta_T$. Therefore Theorem 1 gives

$$d_p\Big(\theta^{(s,k)}, B_{\text{exp}}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\text{exp}}\Big)$$

for every $1 \le p \le \infty$. Chaining this inequality over $k = 1, \dots, T$ and over sweeps $s = 0, \dots, S-1$ gives the final statement. □

2.3 A Second Perspective: Attribution Patching in Parameter Space

The refinement procedure developed in the previous section (Algorithm 1) can be motivated from a second independent starting point that arrive at the same per-coordinate update, comin fron an interpretability-style argument: it treats each move toward a task expert as a parameter-space intervention and uses a first-order attribution-patching score to decide, coordinate by coordinate, whether that intervention is locally loss-decreasing.

The attribution step is a local calculation around the current model state. To make the sequential update rule explicit, let ϑ denote the current merged encoder immediately before applying a single task update. For task i, the task expert θ_i is a known reference solution, and the merge-to-expert direction is

$$v_i(\vartheta) = \theta_i - \vartheta. \tag{14}$$

The corresponding parameter-space intervention is the path

$$\Gamma_i(\alpha; \vartheta) = \vartheta + \alpha v_i(\vartheta), \qquad \alpha \in [0,1], \tag{15}$$

where $\alpha = 0$ is the current model and $\alpha = 1$ is the task expert. Let

$$g_i(\vartheta) = \nabla_\theta \mathcal{L}_i(\vartheta; \mathcal{D}_i^{\text{probe}}) \tag{16}$$

be the task gradient at the current model, evaluated with task i's head. A first-order attribution-patching approximation to the loss change caused by moving from ϑ toward expert i is

$$\Delta_i^{\text{AP}}(\vartheta) = \langle g_i(\vartheta), v_i(\vartheta) \rangle = \sum_{r=1}^{d} a_{i,r}^{\text{AP}}(\vartheta), \tag{17}$$

with coordinate-wise contribution

$$a_{i,r}^{\text{AP}}(\vartheta) = g_{i,r}(\vartheta) v_{i,r}(\vartheta). \tag{18}$$

A negative contribution means that moving coordinate r toward the task expert is locally predicted to reduce task i's loss. A positive contribution means that the same movement is locally predicted to increase the loss. Thus the loss-decreasing expert gate is

$$m_{i,r}(\vartheta) = \mathbf{1}\Big\{ a_{i,r}^{\text{AP}}(\vartheta) < -\epsilon_{\text{gate}} \Big\}, \tag{19}$$

where $\epsilon_{\text{gate}} \geq 0$ is an optional confidence threshold; the default method uses $\epsilon_{\text{gate}} = 0$. The condition $g_{i,r}(\vartheta) v_{i,r}(\vartheta) < 0$ is exactly the condition that the expert direction agrees with the negative gradient in coordinate r.

The resulting coordinate-wise update for task i is a distance-scaled negative-gradient step, masked by the attribution-patching sign:

$$\widetilde{u}_{i,r}(\vartheta) = -g_{i,r}(\vartheta) \; |v_{i,r}(\vartheta)| \; m_{i,r}(\vartheta). \tag{20}$$

Whenever $m_{i,r}(\vartheta) = 1$, this update points toward the expert because $-g_{i,r}(\vartheta)$ and $v_{i,r}(\vartheta)$ have the same sign. The factor $|v_{i,r}(\vartheta)|$ makes the step small when the current model is already close to the expert in that coordinate and larger when the coordinate is far from the expert. Importantly, the gate only certifies a local first-order effect for task i at the current model state. It does not by itself prove that the coordinate is harmless for other tasks, so cross-task interference must be measured experimentally.

3 Experimental Protocol

Primary diagnostic setting: RoBERTa GLUE-8. The primary benchmark is GLUE-8: CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, and RTE. We use RoBERTa-base as the default encoder. Each task expert is obtained by fine-tuning from the same pretrained checkpoint with matched hyperparameter budgets. The primary setting merges full fine-tuned encoders and keeps task-specific heads fixed for evaluation. This setting is intentionally described as *oracle-task-id, multi-head encoder merging*; it is a useful and inexpensive diagnostic but not a complete demonstration of single-model deployment.

Proof-of-concept order. Development proceeds from small to large: a distilled-encoder smoke test on MRPC/STS-B (Sanh et al., 2019), a RoBERTa-base three-task study (MRPC/STS-B/SST-2: one related sentence-pair pair plus a less-related single-sentence control), and the full GLUE-8 scale-up with all feasible baselines, where CoLA and RTE require multiple seeds and MNLI dominates the compute budget. A systematic pairwise and subset study (all pairwise merges, related/stress subsets, the entailment cluster) remains planned as a guard against favorable task selection; distilled models are used only for debugging, never as evidence.

Secondary benchmark settings. Three secondary tracks address the limitations of multi-head GLUE. A shared-output NLP track evaluates GLUE in text-to-text format with T5-style models, so the merged model uses a common generation head rather than task-specific classifiers (Wang et al., 2019; Raffel et al., 2020) (results in Section 4.2). A CLIP/ViT vision track evaluates the standard task-vector image-classification suite—SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD—testing a different modality, different loss geometry, and stronger domain shifts (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024), together with its twenty-task extension (Wang et al., 2024) (adding CIFAR-10/100, STL10, Flowers102, Oxford Pets, PCAM, FER2013, EMNIST, Food101, FashionMNIST, KMNIST, RenderedSST2), which probes the high-interference regime in which merging degrades most (results in Section 4.3). A decoder-only LLM track adopts the MergeBench setting (He et al., 2025): published full-parameter domain experts for the Llama and Gemma families share a common pretrained checkpoint, so task vectors and merge-to-expert directions are well defined and no expert training is required on our side. We begin at a full-fine-tuning-feasible scale (Llama-3.2-3B) with the strongly interfering instruction/mathematics/coding triple—the worst-task-retention stress regime in which the gate helped most on CLIP/ViT—using judge-free automatic metrics (verifiable instruction constraints, exact match on grade-school mathematics, functional-correctness pass rates on code) and subsampled evaluation sets during hyperparameter search, since generation-based evaluation is the dominant cost.

Replay data. For refinement, we use $n \in \{4, 8, 16, 32, 64, 128\}$ replay examples per task. Replay examples are sampled from training data or from a held-out split separated from the final validation split. For classification tasks, replay buffers should be class-balanced whenever possible; for imbalanced tasks such as QQP and for small tasks such as RTE, we report the exact sampling procedure. We report sensitivity to the number of replay examples, variation across replay samples, and the area under the replay-budget curve rather than only the best tuned replay size.

Baselines. Because the proposed method uses labeled replay buffers, baselines are grouped by data regime: *checkpoint-only* (pretrained encoder, single-task experts, uniform averaging, task arithmetic, Fisher merging, TIES, DARE, DELLA-style sampling, Model Breadcrumbs, Localize-and-Stitch); *data-dependent but label-free* (RegMean, AdaMerging, APL); *labeled-replay* (ordinary replay gradient descent from the same initialization, distance-scaled descent without the gate, head-only calibration, encoder and encoder-plus-head replay fine-tuning, multitask replay fine-tuning from the pretrained checkpoint and from the merge); and *sanity controls* (inverted gate, density-matched random gate, wrong-expert directions, shuffled labels, frozen first-step gates, and the aggregated-U variant). All labeled-replay baselines use the same replay examples, the same number of gradient evaluations where possible, and the same hyperparameter search budget—essential because the strongest alternative explanation for a positive result is simply that the method performs supervised post-merge replay fine-tuning.

Metrics. For each GLUE task, we report the standard validation metric: Matthews correlation for CoLA, accuracy for SST-2, matched/mismatched accuracy for MNLI, correlation for STS-B, and accuracy/F1-style scores for the paraphrase and question-pair tasks according to the GLUE convention. Raw task scores, expert scores, pretrained scores, task-arithmetic scores, and absolute drops from the corresponding expert are primary. Because correlation-style metrics can be near zero or negative, the normalized retention metric is

$$\mathrm{NormRet}_t = \frac{\mathrm{Score}_t(\theta_{\mathrm{merge}}) - \mathrm{Score}_t(\theta_0)}{\mathrm{Score}_t(\theta_t) - \mathrm{Score}_t(\theta_0) + \epsilon_{\mathrm{metric}}}, \tag{21}$$

where ϵ_{metric} prevents division by very small denominators. We report mean normalized retention, worst-task normalized retention, harmonic mean normalized retention, and expert-relative drop. Worst-task retention is important because a merge can improve average score while catastrophically harming a small task such as RTE or CoLA. For each main table, we also report confidence intervals across expert fine-tuning seeds and replay-buffer seeds.

Task-similarity and interference analysis. We group GLUE-8 into interpretable clusters—paraphrase/similarity $\{\text{MRPC}, \text{QQP}, \text{STS-B}\}$, entailment $\{\text{MNLI}, \text{QNLI}, \text{RTE}\}$, single-sentence $\{\text{CoLA}, \text{SST-2}\}$—and test whether the gated refinement helps most in related clusters, where movement toward one expert is more likely to be compatible with the others. Planned diagnostics include task-vector cosine similarity, attribution-sign agreement, and the cross-task interference matrix $C_{j \leftarrow i}^{(s,k)} = \langle \nabla_\theta \mathcal{L}_j(\theta^{(s,k-1)}; \mathcal{D}_j^{\text{probe}}), u_i^{(s,k)} \rangle$, whose sign indicates whether task i's update is locally predicted to help or hurt task j.

3.1 Ablations and Diagnostics

The planned ablations cover: **gate sign and necessity** (proposed gate vs. no gate, inverted gate, and a density-matched random mask); **gate confidence and granularity** (ϵ_{gate}, coordinate vs. tensor or block gates, top-k attribution masks, sign-stability masks); **the sequential rule** (immediate vs. aggregated-U; fixed, cyclic, and random task order); **distance scaling** (ordinary replay GD, ungated distance-scaled descent, direct interpolation $u_i \propto m_i v_i$, full gated update); **clipping and normalization** (none, tensor-wise RMS, global-norm clipping, validation line search); **replay budget and variance**; **refinement horizon** $S \in \{0, 1, 2, 5, 10\}$; **expansion point** (fresh gates every update vs. frozen first-sweep gates); and **label/expert sanity checks** (shuffled replay labels, wrong-expert directions).

3.2 Hypotheses

The project evaluates five hypotheses:

H1: Attribution-patching signs predict useful expert directions.
Coordinates with $g_{i,r} v_{i,r} < 0$ should be more likely to reduce task i's loss when moved toward expert i than those with $g_{i,r} v_{i,r} > 0$.

H2: Expert-guided replay refinement improves task arithmetic.
From the same initialization and replay budget, the gated update should outperform ordinary and ungated distance-scaled replay gradient descent.

H3: Sequential single-task updates improve worst-task retention.
Immediate application should reduce overshoot and stale-gradient effects relative to aggregated-U.

H4: Benefits are larger for related tasks.
Gains should concentrate in related clusters, especially MRPC/QQP/STS-B.

H5: Gains are not GLUE-specific.
A positive GLUE result should transfer at least partially to a shared-head text-to-text setting, to CLIP/ViT, or to MergeBench decoder merging.

Table 1: Three-task RoBERTa-base proof of concept (MRPC, STS-B, SST-2; $n = 64$, $S = 5$). Mean normalized retention; best learning rate per family. Left: best configuration, mean ± std over five seeds. Right: a single-seed contrast at a large fixed learning rate, isolating the gate's effect.

Method	Best mean NormRet (± std)	at $\eta = 64$ (1 seed)
Task-arithmetic merge	0.558	—
Ordinary replay GD	0.831 ± 0.005	—
Ungated distance-scaled (no gate)	0.835 ± 0.017	-1.07 (diverges)
APR (gated, proposed)	$\mathbf{0.840 \pm 0.021}$	**0.849** (stable)

4 Preliminary Results

We report initial experiments in the RoBERTa-base multi-head setting (oracle task identity at evaluation). Unless noted, refinement uses S sweeps from the task-arithmetic merge with the corrected trust region applied *after* the learning rate (Eq. (11)) and threshold $\epsilon_{\text{gate}} = 0$. Each method family (ordinary replay gradient descent, the proposed gated update, and the ungated distance-scaled update) is given its own learning-rate search of equal size; we report the best configuration per family by mean normalized retention (Eq. (21)). Section 4.1 additionally compares the refinement against interference-aware and label-free merge baselines, and composes it with them. Section 4.2 then removes the multi-head assumption entirely, repeating the core comparisons on a text-to-text T5 track in which every task decodes through a single shared generation head.

These results are preliminary in three respects. First, the hyperparameter-sweep cells are single-seed, with a run-to-run spread of roughly ±0.02 mean retention (Table 2), so sweep-level differences smaller than that should not be read as orderings; the headline configurations are replicated over five replay-buffer seeds (Table 6), and we flag explicitly the one single-seed conclusion that did not survive replication. Second, hyperparameters (learning rate, and in Section 4.1 also λ, sparsity, and density) are selected *per family* by mean normalized retention on the evaluation split—a uniform and therefore fair rule, but oracle selection; the stricter protocol, in which every labeled method selects its hyperparameters on the replay buffer alone, is applied in Table 9. Third, the multi-head GLUE setting assumes oracle task identity.

Three-task proof of concept. On the MRPC/STS-B/SST-2 triple ($n = 64$ replay examples per task, $S = 5$), all refinement families recover most of the gap from the task-arithmetic merge (0.558) toward the experts and are statistically tied at their tuned optima (Table 1). The decisive difference is learning-rate robustness: the gated update peaks at a much larger step ($\eta = 64$) and remains stable there, whereas the ungated update—identical except for the gate—*diverges* at the same learning rate (-1.07). The gate acts as a protective mask that makes large steps safe by suppressing locally loss-increasing expert directions.

GLUE-8 scale-up. Merging all eight experts (one consistent RoBERTa-base fine-tuning source) is far more interference-prone: the task-arithmetic merge attains only 0.320, with MRPC driven below the pretrained baseline. Expert anchoring matters more here: both anchored updates clearly outperform ordinary replay gradient descent, especially on worst-task retention (Table 2). The gate is approximately performance-neutral relative to the ungated update on mean retention—the ordering flips between the two budgets and the gaps are within single-seed noise—but it again widens the safe learning-rate range (stable through $\eta = 16$, versus an ungated peak at $\eta = 2$–4 and

Table 2: GLUE-8 (all eight tasks, RoBERTa-base, single seed). Best mean normalized retention per family (worst-task retention in parentheses); the task-arithmetic merge baseline is 0.320 (worst −0.175). APR is reported at its largest stable learning rate, $\eta = 16$; the $\eta = 32$ configuration reached 0.614 once but is not reproducible (see text).

Method	n=64, S=5	n=32, S=10
Ordinary replay GD	0.574 (0.04)	0.588 (0.04)
Ungated distance-scaled (no gate)	**0.637** (0.15)	0.607 (0.05)
APR (gated, proposed)	0.595 (0.11)	**0.617** (0.05)

divergence by $\eta \gtrsim 8$–16). MRPC and CoLA remain the persistent worst tasks; QQP, STS-B, MNLI, and QNLI recover to 0.78–0.86.

A caveat on the boundary of that safe range. At $\eta = 32$ the gated update attained 0.614 (worst 0.142) in one run, but an identically configured re-run collapsed to −0.093 (worst −3.467, MRPC); we therefore report the stable $\eta = 16$ configuration. The replay gradients are deterministic (computed in `eval` mode, so no dropout noise enters), which localises the cause to floating-point non-associativity on the GPU: atomics in the embedding backward pass and state-dependent kernel selection perturb the gradient in its last bits, the gate *discretises* the perturbation (coordinates with $g_{i,r}v_{i,r} \approx 0$ flip sign), and near the divergence boundary the trajectories separate. The trust region guarantees that large steps remain *bounded* (Section 2.2), but near the critical learning rate it does not make the outcome *reproducible*; reported learning rates should be interior to the stable range, not at its edge.

What the attribution gate is and is not doing. Two diagnostics temper the interpretation of the gate. First, it keeps a strikingly stable ≈ 36% of coordinates across sweeps, tasks, and learning rates, and a control attributes most of this to gradient sparsity rather than expert geometry: roughly 31% of encoder coordinates are word-embedding rows for tokens absent from the replay buffer (exactly zero gradient, always masked), and a random direction of matched magnitudes reproduces almost the same density (0.345 vs. 0.361 for the true expert direction). Second, the trust-region clip is almost never active at useful learning rates ($< 0.1\%$ of gated coordinates)—a rare-event safeguard, not a routine operation. On GLUE the gate's measurable benefit is therefore learning-rate robustness rather than a higher peak score.

A density-matched random mask makes this concrete. Over five replay-buffer seeds on GLUE-8 (Table 6), the true gate (0.542 ± 0.062) is statistically indistinguishable from—indeed nominally worse than—a random gate of the same density (0.568 ± 0.031), and both trail the ungated update (0.593 ± 0.040): on this modality the gate's *sign* carries no signal beyond its density, as the embedding-sparsity account predicts. The same control behaves oppositely on CLIP (Section 4.1), sharpening the modality split: the attribution sign is informative where gradients are dense, and vacuous where the replay buffer leaves most coordinates without gradient.

CLIP/ViT vision track. We next test transfer to the standard eight-task CLIP task-vector suite (SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD) with a ViT-B/32 backbone: the image encoder is merged while each task keeps a *frozen* zero-shot head derived from the CLIP text encoder; the experts are publicly available fine-tuned encoders, merged with uniform $\lambda_i = 0.3$ (later shown over-scaled; Section 4.1). Refinement uses $n = 64$, $S = 5$, $\gamma = 1$ (single seed). This is a stronger test than GLUE—a different modality, dense gradients without the

Table 3: Eight-task CLIP ViT-B/32 suite (single seed, $n = 64$, $S = 5$, $\gamma = 1$, uniform $\lambda_i = 0.3$). Best learning rate per family by mean normalized retention (Eq. (21)); worst-task retention in parentheses. The gated update is best on both, and is the only method keeping worst-task retention positive.

Method	Mean NormRet	Worst-task
Task-arithmetic merge	0.270	−0.527
Ordinary replay GD	0.458	−0.203
Ungated distance-scaled (no gate)	0.536	−0.086
APR (gated, proposed)	**0.598**	**+0.159**
Inverted-gate control	−1.682	−5.386

Table 4: Per-task top-1 accuracy on the eight-task CLIP suite: zero-shot backbone, task expert, task-arithmetic merge, and the proposed gated refinement (APR, $\eta = 8$). Refinement recovers most of the merge's interference loss, including on Cars and SUN397, which the merge had degraded below the zero-shot backbone.

	SUN397	Cars	RESISC	EuroSAT	SVHN	GTSRB	MNIST	DTD	Avg.
Zero-shot	0.632	0.597	0.582	0.450	0.315	0.325	0.482	0.439	0.478
Expert	0.749	0.783	0.949	0.991	0.972	0.989	0.996	0.794	0.903
Merge	0.570	0.553	0.628	0.734	0.778	0.685	0.961	0.472	0.673
APR	**0.650**	**0.648**	**0.769**	**0.923**	**0.874**	**0.851**	0.961	**0.578**	**0.782**

word-embedding sparsity above, larger domain shifts—and the merge is weak here (0.270 mean, −0.527 worst; SUN397 and Cars fall *below* the zero-shot backbone). All three families recover substantial ground, but—in contrast to GLUE-8—the gate attains the best mean retention and, *among the three replay-refinement families*, the only positive worst-task retention (Table 3); the latter does not extend to merge methods generally, since tuned task arithmetic, Breadcrumbs, and TIES all achieve it with no data (Section 4.1). The gated update lifts the merge from 0.270 to 0.598 mean and from −0.527 to +0.159 worst-task, and the gains concentrate exactly on the interference-damaged tasks the merge drove below the zero-shot floor (Cars and SUN397; Table 4), which the ungated update instead sacrifices at its larger steps (worst-task −2.97 at $\eta = 8$). An inverted-gate control diverges (mean −1.68). Error bars and a second control make this solid (Table 6): over five replay-buffer seeds APR attains 0.605 ± 0.008, against 0.547 ± 0.007 ungated and 0.460 ± 0.002 ordinary GD, while a density-matched *random* gate collapses to 0.498 ± 0.117 with wildly unstable worst-task retention (-0.564 ± 0.888)—on CLIP the attribution sign is the signal. A smaller three-task vision study (EuroSAT/GTSRB/MNIST, single seed) shows the same ordering more sharply (APR 0.940 mean / 0.915 worst, vs. 0.917/0.876 ungated and 0.874/0.792 ordinary GD), and as on GLUE the best learning rate falls as tasks are added. Together these results support H5 and establish that the gate's usefulness is *modality-dependent*: approximately neutral on GLUE encoders, beneficial on both mean and worst-task retention on CLIP/ViT. Section 4.3 sharpens this further: on the twenty-task extension of the same modality the gate's advantage over the ungated anchored update washes out again, so the dependence is better described as *interference-regime-dependent* than modality-dependent.

Replay budget and refinement horizon (CLIP/ViT). Two further eight-task studies probe the replay budget and the refinement horizon against ordinary replay gradient descent, the strongest

Table 5: Replay-budget study on the eight-task CLIP suite (mean normalized retention; worst-task in parentheses). Best configuration per family; the gated update (APR) near-converges by $S = 5$–10 sweeps, while ordinary replay GD is given a longer horizon ($S = 25$–35) as a generous baseline. APR holds an $\approx +0.10$ mean advantage at every budget and is the only method with non-negative worst-task retention down to $n = 16$.

Replay budget	**APR (ours)**	Ordinary replay GD
$n = 16$	**0.522** (+**0.018**)	0.427 (−0.266)
$n = 32$	**0.570** (+**0.122**)	0.476 (−0.197)
$n = 64$	**0.598** (+**0.159**)	0.526 (−0.135)

baseline that also uses labels. Reducing the buffer from $n = 64$ to 32 and 16 degrades both methods gracefully, but the gated update keeps an almost constant $\approx +0.10$ mean advantage at every budget and is the only method with non-negative worst-task retention down to $n = 16$ (Table 5); its best learning rate falls as the buffer shrinks (from $\eta = 8$ at $n = 64$ to $\eta = 6$ at $n \leq 32$), consistent with noisier gradients favoring smaller trust-region steps. On the horizon, the gated update is essentially converged after five sweeps ($0.598 \to 0.615$ from $S = 5$ to $S = 15$ at $n = 64$, with no sign of overfitting), whereas ordinary gradient descent is still improving at $S = 35$ ($0.458 \to 0.504 \to 0.526$ at $S = 5, 15, 35$) yet never catches up and never lifts its worst task (SUN397) back above the zero-shot backbone. The expert anchor, rather than additional data or steps, is what recovers the most damaged tasks.

4.1 Merge baselines, and composition with them

Every experiment above starts from a uniform task-arithmetic merge, leaving the central objection unanswered: perhaps the refinement only repairs a weak initialization, and a better *merge* would make it unnecessary. We therefore evaluate the refinement against, and on top of, the merge methods it is supposed to complement (Table 8): checkpoint-only baselines using no data (tuned task arithmetic, TIES (Yadav et al., 2023), DARE (Yu et al., 2023), DARE-TIES, Model Breadcrumbs (Davari and Belilovsky, 2023)) and label-free data-dependent baselines (AdaMerging (Yang et al., 2024), which learns task- or layer-wise coefficients by entropy minimization, and RegMean (Jin et al., 2023), which merges each linear layer in closed form from input Gram matrices; RegMean gets the same 64 unlabeled inputs per task that APR sees labeled, and a 256-input variant changes its scores by less than 0.02). All families share the same experts, replay buffers, frozen heads, and per-family tuning rule.

The refinement composes with better merges rather than replacing them. On CLIP-8 APR improves *every* merge initialization it is given, and the best results always come from the strongest initializations, never plain task arithmetic: it lifts AdaMerging from 0.610 to **0.722** mean / +**0.429** worst-task and RegMean from 0.733 to 0.762. On GLUE-8 the same pattern holds with one instructive exception analysed below: APR lifts DARE-TIES from 0.301 to 0.610, but *harms* the RegMean initialization at every learning rate. The refinement is complementary to good merging, not an alternative—and because the matched-budget AdaMerging initialization uses only the same 64 inputs per task that APR sees labeled, that composition consumes no data beyond the replay buffer and never touches the evaluation split.

Table 6: Multi-seed replication of the headline configurations (five replay-buffer seeds; mean ± std of mean normalized retention, worst-task in parentheses). Each cell re-runs the best configuration from the corresponding sweep with a freshly sampled replay buffer per seed; TIES is deterministic given the checkpoints and is shown for reference. The random-gate control uses a random mask of the same density as the attribution gate.

Method	CLIP-8 mean ± std (worst ± std)	GLUE-8 mean ± std (worst ± std)
TIES (deterministic)	0.528 (+0.255)	0.302 (0.000)
Ordinary replay GD ← TA	$0.460 \pm .002$ ($-0.200 \pm .007$)	$0.483 \pm .017$ ($-0.048 \pm .058$)
Random gate ← TA	$0.498 \pm .117$ ($-0.564 \pm .888$)	$0.568 \pm .031$ ($+0.030 \pm .043$)
Ungated ← TA	$0.547 \pm .007$ ($-0.068 \pm .016$)	$0.593 \pm .040$ ($+0.085 \pm .096$)
AdaMerging (layer-wise)	$0.595 \pm .037$ ($+0.168 \pm .076$)	—
APR ← TA	$0.605 \pm .008$ ($+0.151 \pm .020$)	$0.542 \pm .062$ ($-0.170 \pm .430$)
APR ← TIES	$0.635 \pm .061$ ($+0.151 \pm .399$)	—
APR ← DARE-TIES	—	$0.620 \pm .021$ ($+0.180 \pm .053$)
APR ← AdaMerging	$\mathbf{0.699 \pm .044}$ ($+0.310 \pm .223$)	—

Error bars, and a single-seed conclusion retracted. Replicating the headline configurations over five replay-buffer seeds (Table 6) mostly confirms the single-seed picture, with one exception we retract explicitly. In a single-seed sweep, layer-wise AdaMerging (0.610) appeared to edge out APR from the tuned task-arithmetic merge (0.588) on CLIP-8 despite using no labels; at five seeds the ordering is statistically indistinguishable and nominally *reversed* (APR 0.605 ± 0.008 vs. AdaMerging 0.595 ± 0.037), so we treat the two as tied. Two findings survive. First, AdaMerging is remarkably data-efficient: restricting it from its standard transductive protocol to exactly the 64 unlabeled inputs per task that APR sees labeled does not degrade it at all ($0.604 \rightarrow 0.610$ mean; $+0.165 \rightarrow +0.219$ worst). Second, that a label-free method *ties* a labeled one from the same initialization demands explanation, which we provide next.

Why is a label-free method competitive with a labeled one at all? The answer appears to be parameterization rather than information. AdaMerging does not merely rescale the merge: the spread of λ_i^l across tensors within a task (0.151) is comparable to its mean (0.258), attention projections are amplified to nearly twice the initial coefficient while embedding and bias contributions are driven toward zero, and 38% of coefficients *increase*. The dominant recoverable error in the CLIP merge is thus a per-tensor mixing error, living in a well-conditioned space of $\sim$1600 coefficients—exactly the space AdaMerging searches, with a joint objective over all tasks. APR is poorly suited to it for optimization rather than reachability reasons: it alternates greedy single-task steps and never evaluates the cross-task trade-off that re-mixing requires, and realizing a coherent per-tensor rescaling as the net effect of $\sim 10^8$ independently gated coordinate moves, each sized by a 64-example gradient, is a far noisier estimator of the same $\sim$1600 numbers than fitting them directly. Consistent with this, the *task-wise* AdaMerging variant (eight degrees of freedom) scores 0.344, worse than a single tuned global coefficient (0.407): the useful knowledge is which tensors to re-mix, not which tasks to weight. The diagnosis predicts that giving a *labeled* objective the same low-dimensional per-tensor degrees of freedom should recover AdaMerging's advantage while avoiding its failure mode on trained heads—a direction we leave to future work.

Coefficient tuning corrects an earlier conclusion. The uniform $\lambda_i = 0.3$ used throughout the CLIP experiments above is substantially over-scaled: $\lambda_i = 0.2$ raises the merge from 0.270 / -0.527

Table 7: Long-horizon control: do more sweeps, random task order, and cosine learning-rate annealing improve the refinement? Mean normalized retention (worst-task in parentheses); "anneal" = S=30, random order per sweep, cosine decay to 5% of the peak learning rate. CLIP-8 columns are single-seed; GLUE-8 columns are mean ± std over five replay seeds (the steps-only row was run single-seed and is shown for CLIP only). At eight tasks the refinement is near-converged by S=5 on CLIP, whereas on GLUE the annealed long horizon is clearly the best and most stable configuration.

	CLIP-8 (1 seed)		GLUE-8 (5 seeds)	
Method	n=64	n=32	n=64	n=32
APR, S=5 (baseline)	0.598 (0.16)	0.551 (0.02)	$0.542 \pm .062$	$0.554 \pm .068$
APR, S=30, fixed, constant	**0.616** (0.13)	0.570 (0.12)	—	—
APR, S=30, anneal	0.604 (0.13)	**0.578** (0.12)	$\mathbf{0.641 \pm .028}$	$\mathbf{0.596 \pm .032}$

to 0.407 / +0.069. As noted in Section 4, this invalidates any reading of Table 3 in which APR is the only method with positive worst-task retention on CLIP-8; tuned task arithmetic, Breadcrumbs, and TIES all achieve it using no data. On GLUE-8, by contrast, $\lambda = 0.3$ is already the interior optimum of the same grid (0.175, **0.320**, 0.239, 0.143 for $\lambda \in \{0.2, 0.3, 0.4, 0.5\}$), so the GLUE comparisons above are unaffected.

Entropy is a good surrogate for accuracy only when the heads are frozen. AdaMerging's usefulness is modality-dependent in the opposite direction to the gate's. On GLUE-8 it drives its unlabeled objective down hard—mean entropy 0.610 → 0.163 for the layer-wise variant—yet reaches only 0.173 mean retention, far *below* the 0.320 merge it starts from, and the task-wise variant collapses outright (0.023; worst-task −3.068, MRPC). The explanation is structural: CLIP's frozen zero-shot heads can only become confident through better image features, whereas RoBERTa's trained classifiers let the encoder be dragged into whatever region makes them confident—not the region that makes them correct—so minimizing entropy makes the merged model confidently wrong. This cuts against the intuition that a label-free objective which works on one benchmark is a safe default, and it is precisely the failure mode a *labeled* replay signal cannot exhibit.

Does a longer, annealed schedule improve the refinement? A natural question is whether single-stage APR is under-optimized at its default setting of $S = 5$ sweeps, fixed task order, and a constant rate—perhaps more sweeps, order randomization, and a decaying step would help. We test this directly (Table 7): $S = 30$ with re-randomized task order and cosine decay to 5% of the peak rate, at two peaks, plus a steps-only control ($S = 30$, fixed order, constant rate), at $n = 64$ and $n = 32$ (single seed on CLIP-8, five replay seeds on GLUE-8). The answer is modality-dependent. On CLIP-8 the whole schedule package buys only +0.006 to +0.027 over the $S = 5$ baseline, never exceeds 0.616, and does not reliably beat the steps-only control: at eight tasks the refinement is essentially converged after five sweeps. On GLUE-8 the schedule matters much more—over five replay seeds the annealed long refinement is the *best and most stable* GLUE configuration at both budgets (0.641 ± 0.028 at $n = 64$; 0.596 ± 0.032 at $n = 32$), well above the $S = 5$ baseline (0.542 ± 0.062). The twenty-task CLIP results of Section 4.3 qualify the CLIP half of this conclusion: convergence by $S = 5$ is a property of the eight-task regime, and at twenty tasks the longer annealed horizon becomes worth roughly +0.01 mean and a large gain in worst-task accuracy.

Which ingredient of the sparsification baselines matters. DARE alone reproduces task arithmetic almost exactly (CLIP 0.270 vs. 0.270; all nine sampled cells fall in $[0.249, 0.272]$), as

it must: its drop-and-rescale is expectation-preserving, so summed over $\sim 10^8$ coordinates the merge concentrates back onto the unsparsified one. It is therefore reported as a control rather than a competitor, and it isolates *sign election*—not sparsification—as the operative ingredient of TIES. Pairing the two (DARE-TIES) is scale-sensitive, because DARE's 1/density rescale inflates magnitudes before the trim: at trim density 0.1 it is well behaved, and at 0.2 it collapses MRPC.

The optimal step size depends on the initialization—and boundedness is not usefulness. APR's raw update and its trust region both scale with the expert distance $|v_{i,r}| = |\theta_{i,r} - \theta_r|$, which differs across merge points, and the two roles of that scale are worth separating. The clip binds exactly when $\eta\,|g_{i,r}| > \gamma$—the $|v_{i,r}|$ cancels—so the learning rate controls *what fraction* of coordinates saturate, while the initialization's distance controls *the absolute length* of every saturated move ($\gamma|v_{i,r}|$: the whole way to the expert). From a distant initialization the same η therefore produces the same saturation fraction but vastly larger physical steps: at $\eta = 8$ we measure first-sweep step norms of 0.13–2.96 from the RegMean merge versus 0.035–0.13 from the task-arithmetic merge, a 4–30× difference at identical gate density. In the saturated regime the dynamics remain *bounded*—each task's step still cannot overshoot its own expert, and we verified convergence up to $\eta = 10^6$—but they converge to a coordinate-wise patchwork of the eight experts, overwritten task by task, whose quality is close to the unrefined merge. Boundedness is a tuning-robustness property; usefulness requires steps that are perturbative relative to the initialization's structure. Re-centering the grid accordingly recovers interior optima: APR from RegMean peaks at $\eta = 0.5$ on CLIP-8 ($0.733 \to 0.762$), thirty times below the task-arithmetic optimum. Measuring the geometry confirms the same mechanism behind the earlier TIES failures: relative to task arithmetic, the TIES merge is *closer* to six of the eight GLUE experts but substantially farther from the two large-data ones (QQP, MNLI), so at a fixed η those two tasks take steps 1.6–2.2× larger and trample the fragile MRPC; in every failed cell exactly one fragile task collapses (MRPC on GLUE, Cars on CLIP).

One exception is not a step-size effect and delimits when composition works. On GLUE-8, APR harms the RegMean initialization at *every* learning rate, including deeply perturbative ones ($0.478 \to 0.103$ at $\eta = 0.25$). Two ingredients coincide there: the gate is uninformative on GLUE (no better than random; Table 6), and RegMean's least-squares solution is exactly the point that *deviates* from the task-vector directions to cancel interference—so any movement back toward the experts, however small, undoes the fit. Composition with APR therefore requires (i) a gate that carries signal on the modality, and (ii) an initialization whose structure survives movement toward the experts; task arithmetic, TIES, and AdaMerging satisfy (ii), while a regression optimum does not.

Matched labeled budget: the same 64 examples, every method. The comparison so far spans the checkpoint-only and label-free regimes, but the method lives in the small-labeled-data regime, and the sharpest test holds the data fixed and varies only the method. Table 9 gives every labeled method exactly APR's $n = 64$ replay examples per task, drawn from the training split, and—unlike the tables above—selects each method's hyperparameters on the *replay buffer* rather than the evaluation split (the strict protocol of Section 4); we note where this replay-selected number differs from the evaluation-selected one. Against this budget, alternative uses of the same labels are surprisingly weak. Tuning a single global λ recovers the task-arithmetic point (it is already optimal); tuning per-task λ_i by coordinate descent *overfits* the buffer, collapsing to 0.144 on GLUE-8; LM-Cocktail-style loss weighting (Xiao et al., 2024) and Fisher merging (Matena and Raffel, 2022) with a diagonal Fisher estimated from 64 examples are both weak on both suites. Localize-and-Stitch (He et al., 2024), the closest published relative of the attribution gate, is the

Table 8: Merge baselines and composition (single seed, $n = 64$, $S = 5$). Mean normalized retention, worst-task in parentheses. Every family shares the same experts, replay buffers, and heads; hyperparameters (λ, density, sparsity, η) are tuned per family on the evaluation split (see Section 4). ‡: standard *transductive* AdaMerging draws unlabeled inputs from the evaluation split; the CLIP rows instead use the matched budget (the same 64 unlabeled inputs per task that APR sees labeled), which *improves* on transductive (0.333 / 0.604 for task/layer wise), and the matched layer-wise merge is the initialization for the last row (making that composition fully inductive). On GLUE both budgets fail alike (transductive 0.023 / 0.173; matched 0.029 / 0.198 for task/layer wise). RegMean uses the matched 64 unlabeled inputs per task (a 256-input variant scores 0.747 / 0.483). ¶: APR harms the RegMean initialization on GLUE-8 at every learning rate (see text); the value shown is its least-bad cell. **: favorable seed—across five replay seeds this cell is 0.480 ± 0.153 at its fixed refinement rate (Table 7 and accompanying text); the GLUE column's bold entry is the most robust composition instead. Multi-seed replications of the headline cells appear in Table 6.

Method	CLIP-8 mean (worst)	GLUE-8 mean (worst)
Checkpoint-only (no data)		
Task arithmetic, $\lambda_i = 0.3$	0.270 (−0.527)	0.320 (−0.175)
Task arithmetic, λ tuned	0.407 (+0.069)	0.320 (−0.175)
DARE (control)	0.270 (−0.524)	0.337 (+0.009)
DARE-TIES	0.364 (−0.319)	0.301 (−0.344)
Model Breadcrumbs	0.431 (+0.079)	0.143 (−0.267)
TIES	0.528 (+0.255)	0.302 (0.000)
Label-free, data-dependent		
AdaMerging (task-wise)	0.344 (−0.239)	0.023‡ (−3.068)
AdaMerging (layer-wise)	0.610 (+0.219)	0.173‡ (0.000)
RegMean	0.733 (+0.443)	0.478 (+0.178)
Labeled replay ($n = 64$)		
APR ← task arithmetic	0.588 (0.185)	0.595 (0.108)
APR ← Breadcrumbs	0.606 (0.212)	0.523 (0.095)
APR ← DARE-TIES	0.626 (0.190)	**0.610** (**0.178**)
APR ← TIES	0.653 (0.333)	0.571 (0.165)
APR ← AdaMerging	**0.722** (**0.429**)	0.583 (0.050)
APR ← RegMean	0.762 (0.441)	0.103¶ (−0.062)

most instructive comparison. Its *learned* masks—trained on the same replay buffer, with the ℓ_1 sparsity penalty swept over four orders of magnitude and the stitch scale tuned—reach only 0.328 / 0.173, consistently *below* its own data-free magnitude variant (0.520 / 0.415): at this sample size, learning which coordinates to keep is worse than simply taking the largest ones. The competitive labeled baselines are instead the ones that touch many parameters: the magnitude stitch and the encoder-refinement family. On CLIP-8, APR is the best method at this budget (0.598, confirmed at 0.605 ± 0.008 over five seeds); on GLUE-8 the encoder-refinement family dominates but the gate is not the decisive ingredient there (0.542 ± 0.062 for APR versus 0.593 ± 0.040 ungated), consistent with the modality-dependence established above.

Head-only recalibration exhibits the same modality asymmetry as AdaMerging, and for the same reason: refitting each task head on 64 examples over a frozen merged encoder is the second-strongest method on GLUE-8 (0.613) but catastrophic on CLIP-8 (−0.415, with SUN397 and Cars driven far below the zero-shot floor). A trained GLUE classifier head can absorb a small labeled sample;

Table 9: Matched labeled-data budget: every method uses the same 64 labeled examples per task (from train), with hyperparameters selected on the replay buffer, not the evaluation split. Mean normalized retention, worst-task in parentheses. [§]: the evaluation-selected configuration differs from the replay-selected one shown (evaluation-selected values: ungated CLIP 0.536; APR GLUE 0.595; learned Localize-and-Stitch 0.371 / 0.421).

Method ($n = 64$ labeled)	CLIP-8	GLUE-8
Global λ tuned on replay	0.270 (-0.53)	0.320 (-0.18)
Per-task λ_i (coordinate descent)	0.377 (-0.02)	0.144 (-1.96)
LM-Cocktail loss-weighted	0.214 ($+0.04$)	0.074 (-0.03)
Fisher merging (replay Fisher)	0.231 (-0.05)	-0.010 (-1.16)
Head-only recalibration	-0.415 (-4.30)	0.613 ($+0.13$)
Localize-and-Stitch (learned masks)	0.328[§] (-0.10)	0.173[§] (-1.10)
Localize-and-Stitch (magnitude, data-free)	0.520 ($+0.22$)	0.415 ($+0.02$)
Ordinary replay GD	0.458 (-0.20)	0.504 (-0.02)
Ungated distance-scaled	0.521[§] (-0.32)	**0.637** ($+0.18$)
APR (proposed)	**0.598** (**+0.16**)	0.570[§] ($+0.09$)

CLIP's head is a frozen zero-shot anchor, and unfreezing it to fit 64 images destroys the very alignment that makes the merge work. The labeled signal helps through the head only when the head is trainable, which reinforces that APR's gains on CLIP are encoder repair rather than head realignment.

4.2 Shared-output T5 track: merging through a single generation head

Every GLUE result above assumes oracle task identity: the merged encoder is scored with each task's own trained classifier. The shared-output track removes this assumption entirely. We merge full flan-T5-base sequence-to-sequence models—encoder, decoder, and LM head—fine-tuned on the eight GLUE tasks (the FusionBench checkpoints; Raffel et al., 2020; Tang et al., 2024), and every task is evaluated by decoding through the single shared head under FusionBench's text-to-text templates, so nothing task-specific remains in the merged model. The protocol otherwise mirrors the GLUE-8 and CLIP-8 studies: $n = 64$ labeled replay examples per task, $S = 5$ sweeps, $\gamma = 1$ with the corrected trust region, equal-size learning-rate grids per family, single seed, evaluation-selected except where noted. One metric convention is forced by the setting: zero-shot flan-T5-base emits non-numeric strings on the STS-B regression task, so its base correlation is undefined; we set the base score to zero for that task (its retention is measured against a zero floor). The family ordering reported below is unchanged if STS-B is dropped from the average. The uniform coefficient $\lambda_i = 0.3$ is again the interior optimum of the $\lambda \in \{0.2, \ldots, 0.5\}$ grid, as on multi-head GLUE-8.

The multi-head findings carry over without the oracle assumption. Table 10 reproduces, in a setting with a single shared generation head, every qualitative conclusion of the multi-head study. Expert anchoring is again the decisive ingredient: both anchored updates (0.594 ungated, 0.537 gated) clearly outperform ordinary replay gradient descent (0.414) and the merge (0.383), while the gate itself is approximately neutral to slightly negative on mean retention—the third NLP setting in which anchoring, not the gate, carries the mean-retention gain, consistent with the modality-dependence established on GLUE-8. The gate's *sign* still carries the signal: the inverted-gate control diverges catastrophically (-6.185). Composition behaves exactly as on the other suites: APR improves every merge initialization it is given, and the best result never comes

Table 10: Shared-output T5 track: flan-T5-base text-to-text GLUE-8 (single seed, $n = 64$, $S = 5$, $\gamma = 1$, $\lambda_i = 0.3$). The whole sequence-to-sequence model including the LM head is merged; every task decodes through the same head, so no oracle task identity is assumed. Mean normalized retention (worst-task in parentheses); best configuration per family. STS-B retention is measured against a zero base (see text). Nearly every refinement family's optimum sits at the *top* of its learning-rate grid (the grids were inherited from the RoBERTa study and shifted down one notch, which turned out too conservative for T5), so the refinement values are lower bounds; see text.

Method	mean (worst)
Checkpoint-only (no data)	
Task arithmetic, λ tuned	0.383 (−0.250)
DARE-TIES	0.391 (−0.417)
Model Breadcrumbs	0.409 (0.000)
DARE (control)	0.427 (0.000)
TIES	0.457 (+0.008)
Labeled replay ($n = 64$)	
Ordinary replay GD	0.414 (0.000)
Ungated distance-scaled	0.594 (+0.031)
APR ← task arithmetic	0.537 (+0.024)
APR ← Breadcrumbs	0.468 (+0.058)
APR ← TIES	0.619 (+0.166)
APR ← DARE-TIES	**0.687** (+0.108)
Inverted-gate control	−6.185 (−16.714)

from plain task arithmetic. Most strikingly, DARE-TIES—the *weakest* checkpoint-only merge here (0.391, worst −0.417)—is the best launch point (**0.687**, worst +0.108), the same rescue pattern as on multi-head GLUE-8, where APR also lifted DARE-TIES from last place to the top of the composition column.

Caveats. Three limitations are explicit. The track is single-seed. Nearly every refinement family's tuned optimum lies at the top of its learning-rate grid ($\eta = 8$ ungated, $\eta = 32$ for the compositions), so the labeled-tier values are lower bounds and the grids need re-centering upward before these numbers are final. And the far edge already shows the familiar warning sign: extending APR from task arithmetic to $\eta = 64$ raises mean retention ($0.537 \to 0.557$) while flipping worst-task retention negative ($+0.024 \to -0.103$)—the same edge-of-stability signature documented for GLUE-8 at $\eta = 32$, so chasing the unbracketed optima will require the same interior-of-the-stable-range discipline used there.

4.3 Scaling to twenty tasks: the high-interference regime

We next scale the vision track to the twenty-task suite of Wang et al. (2024), using the published fine-tuned ViT-B/32 encoders for all twenty tasks (every expert was validated to beat its zero-shot floor by a wide margin, including the three tasks whose class-name orderings we had to reconstruct: PCAM $0.556 \to 0.887$, FER2013 $0.376 \to 0.711$, RenderedSST2 $0.586 \to 0.713$). Unless noted, all results in this subsection are single-seed with $n = 64$ replay examples per task and hyperparameters selected on the evaluation split, symmetrically for every method; a stricter disjoint-holdout selection protocol, piloted at the end of this section, changes the numbers negligibly, and the headline cells are replicated over five replay-buffer seeds (Table 16).

Table 11: Twenty-task CLIP suite: initialization × method at matched budget ($n = 64$, $S = 5$, single seed). Mean top-1 accuracy over the twenty tasks (worst-task in parentheses); zero-shot floor 0.550, expert ceiling 0.896. Best per family by mean accuracy; each family's learning rate is tuned per initialization. The refinement improves every initialization, and APR ≈ the ungated anchored update everywhere (±0.01), so at this scale the anchored, distance-scaled, clipped step—not the gate—is the load-bearing ingredient. The ungated update from RegMean peaked at the low edge of its grid, so that cell is a lower bound. RegMean row from the matched $n = 64$ Gram budget.

Initialization	Merge alone	+ ungated	+ APR
Task arithmetic ($\lambda = 0.08$)	0.604 (0.115)	0.670 (0.203)	**0.680** (0.221)
TIES	0.626 (0.150)	**0.701** (0.277)	0.697 (0.262)
DARE-TIES	0.617 (0.149)	**0.697** (0.277)	0.695 (0.254)
Breadcrumbs	0.611 (0.111)	0.669 (0.168)	**0.678** (0.177)
AdaMerging-layer	0.665 (0.101)	0.701 (0.140)	**0.707** (0.188)
RegMean	0.723 (0.234)	0.756 (0.520)	0.749 (0.486)

Two protocol corrections forced by the scale. First, the uniform $\lambda_i = 0.3$ that is standard at eight tasks is catastrophic at twenty: the task-arithmetic merge scores 0.364 mean top-1 accuracy, *below* the 0.550 zero-shot floor, and the tuned optimum falls to $\lambda \approx 0.05$–0.08 (accuracy 0.604; the curve is monotone over $0.3 \to 0.08$ and flat below). Merge coefficients must be re-tuned per task count, and refinement must never start from a hand-picked global λ. Second, normalized retention (Eq. (21)) becomes *degenerate* at this scale: the suite contains tasks whose expert barely improves on the zero-shot backbone (STL10 gap 0.0043, Food101 gap 0.035), so the denominator explodes—STL10 alone produces per-task retentions near -60 and is the "worst task" of nearly every method for that reason only. On this suite we therefore report *absolute* mean and worst-task top-1 accuracy (zero-shot floor 0.550, expert ceiling 0.896) as primary, with retention restricted to non-degenerate tasks as a secondary check.

An initialization-by-method matrix. At eight tasks, the refinement had only ever been characterized from the task-arithmetic initialization. Table 11 closes that gap: from five initializations (tuned task arithmetic, TIES, DARE-TIES, Breadcrumbs, AdaMerging-layer) plus RegMean, we run APR and the ungated anchored update at matched budget, each with its learning rate tuned per initialization. Two findings. (i) APR and the ungated anchored update are within ±0.01 everywhere—confirmed over five replay seeds (-0.002 ± 0.006 from TIES, $+0.001 \pm 0.005$ from AdaMerging; Table 16): at twenty tasks the gate is neutral on this modality too, and the anchored, distance-scaled, clipped step is the load-bearing ingredient. The attribution *sign* still carries signal—the inverted-gate control collapses to 0.100, far below the zero-shot floor—but thresholding at zero adds nothing over no gate, consistent with the GLUE analysis. (ii) Refinement improves every initialization, and better initializations retain their lead through refinement: the ordering ada > ties ≈ dareties > ta ≈ bc is preserved.

A held-out attribution audit: why thresholding the gate cannot help. The natural response to a neutral gate is to threshold harder—keep only the coordinates with the largest attribution $|g_{i,r} v_{i,r}|$, or (arguing that v is deterministic and g is the only noisy factor, so small-$|g|$ coordinates with large $|v|$ pass the product despite carrying unreliable signs) the largest $|g_{i,r}|$. We tested the premise directly. At the tuned merge, per task, we bucket the $g_{i,r} v_{i,r} < 0$ coordinates by percentile of the selection statistic ($|gv|$, $|g|$, and per-tensor-normalized $|g|$), apply a small

toward-expert step restricted to each bucket, and measure the loss change both on the n-example buffer that produced the gradient and on a *disjoint* held-out buffer of equal size. On the training buffer the first-order prediction is realized almost exactly in every bucket (0.82–0.99 of the predicted decrease at $\alpha = 0.05$)—necessarily so, since any $gv < 0$ coordinate reduces the loss it was measured on; the informative quantity is the *transfer ratio*, held-out over training decrease. Three facts close the question. First, the gradient is indeed noisy: only ≈32% of the first-order training decrease transfers at $n = 64$, and ≈10% at $n = 8$. Second, the noise is *uniform across magnitude buckets*: at $n = 64$ the transfer ratio is 0.31–0.40 from the top 0.1% down to the bottom half, the top 0.1% of coordinates carries only ≈5% of the transferable gain (top 5%: ≈41%), and at $n = 8$ the ordering *reverses*—the bottom half transfers about twice as well as the top 0.1% (0.11 vs. 0.06), because the largest gradient entries on a tiny sample are the most sample-specific. A magnitude threshold would therefore discard most of the transferable signal, and would select the worst-generalizing coordinates exactly in the small-buffer regime where noise is worst; $|g|$ never outperforms $|gv|$, and all three statistics agree to within 0.02. Third, and most tellingly, the sign-control bucket—toward-expert steps on $g_{i,r}v_{i,r} > 0$ coordinates, predicted to *increase* the loss—fails to transfer: its held-out effect is near zero (transfer ratio 0.27 at $n = 64$, 0.01 at $n = 8$, where its held-out sign agreement is 9/20 tasks, i.e. chance). On held-out data, moving toward the experts helps almost regardless of which coordinates are selected, and the gradient's coordinate-level selection carries little information that generalizes. This explains the gate's neutrality mechanistically, closes the thresholding route ($\epsilon > 0$, top-k by either statistic), and locates the gradient's transferable contribution where the tables already measure it: adaptive step magnitude and stopping, i.e. the gap between the coefficient-merge family and the refinement, not coordinate choice.

The refinement is not converged at five sweeps at this scale. Unlike at eight tasks, where the refinement is essentially converged after five sweeps (Table 7), at twenty tasks a longer horizon still pays: running APR to $S = 30$ with a cosine schedule and re-randomized task order improves every initialization we tested, by roughly +0.01 in mean accuracy. The larger effect is on *worst-task* accuracy, which the longer horizon lifts substantially—for example from 0.262 to 0.386 starting from TIES, and from 0.188 to 0.312 from AdaMerging—so that from every checkpoint-space initialization the refined model retains at least 0.31 on its hardest task (Table 12). The fragile-task-recovery property that the $S = 5$ snapshot appeared to lose at this scale is therefore a budget artifact rather than a property of the twenty-task regime. Consistent with the eight-task control, the schedule itself contributes little: the gain comes primarily from the additional sweeps, with cosine annealing worth a further ∼0.005 and, from the base model, also reaching its peak at a *smaller* final displacement. The returns are exhausted by thirty sweeps: extending to $S = 60$ moves the best configuration by at most 0.004 at any budget (see the replay-budget study below).

Hyperparameter robustness. Aggregating *every* APR cell run on this suite (all learning rates, sweep counts, and schedules): of 23 cells, none fell below its own initialization and none fell below the zero-shot floor—the worst cell in the entire campaign scores 0.614, at double the optimal learning rate. The tested safe window spans the full 8× learning-rate grid, and the optimum ($\eta \approx 4$–8) transfers across all checkpoint-space initializations, so a rate tuned at one starting point can be reused at another; only the far RegMean initialization requires its own, much smaller rate. This is the practical content of the trust region: with $\gamma \leq 1$ and clipping applied after the learning rate, no step can overshoot its expert coordinate-wise (Appendix, box-distance monotonicity), so there is no divergent regime to tune around—degradation at excessive rates is gradual rather than catastrophic. The multi-seed replication sharpens this into its strongest form: for the unconstrained-descent probe,

Table 12: Twenty-task CLIP suite: best APR result over *any* refinement budget (learning rate, sweeps $S \in \{5, 30\}$, and constant/cosine schedule tuned per initialization), mean top-1 accuracy with worst-task in parentheses; zero-shot floor 0.550, expert ceiling 0.896; single seed. Initializations ordered by their own accuracy. The refinement improves every starting point, and the gain shrinks monotonically as the initialization improves (+0.13 from the base model, +0.04 from RegMean) while worst-task accuracy at least doubles at every initialization. Refining directly from the base model ($\lambda = 0$, top row) already exceeds the tuned task-arithmetic merge.

Initialization	Init. alone	+ APR (best)
Base model θ_0	0.550 (0.100)	**0.683** (0.350)
Task arithmetic (λ=0.08)	0.604 (0.115)	**0.693** (0.351)
Breadcrumbs	0.611 (0.110)	**0.695** (0.357)
DARE-TIES	0.617 (0.149)	**0.706** (0.374)
TIES	0.626 (0.150)	**0.708** (0.386)
AdaMerging-layer	0.665 (0.101)	**0.712** (0.312)
RegMean	0.723 (0.234)	**0.764** (0.502)

a configuration validated on one buffer draw is not safe on another—three of its twenty pinned runs diverged to near-chance accuracy when *only the replay buffer was redrawn* (replay loss rising, e.g. $0.694 \to 0.132$ from AdaMerging at the rate that was optimal on seed 0), and *which* schedule fails flips between seeds (from RegMean the cosine arm diverged on the seed where the constant arm was fine). The anchored update had zero failures in its twenty runs, with seed standard deviations of 0.002–0.006 throughout. Tuning plain descent once and reusing it is therefore unreliable under nothing more than a buffer redraw; the trust region removes exactly this failure mode.

Composition, again. The refinement composes with the strongest initializations at this scale as it does at eight tasks. RegMean is the best label-free initialization (0.723 at the matched 64-input Gram budget; 256 inputs add only +0.006), and APR lifts it to 0.764 mean / 0.502 worst-task, the best configuration in the twenty-task study. The far-initialization step-size law replicates exactly: APR's optimal rate from RegMean is $\eta = 0.5$ (an interior peak), an order of magnitude below its rate from checkpoint-space merges. AdaMerging must itself be initialized with care at this scale: started at its conventional $\lambda = 0.3$ (inside the collapse region) it climbs to only 0.654 in 300 steps, while started at $\lambda = 0.05$ it reaches 0.665; both learned endpoints agree on $\bar{\lambda} \approx 0.1$, approached from opposite sides.

RegMean is also the initialization at which the anchored update is least advantaged, and the reason is specific to how RegMean sits in parameter space. RegMean solves each linear layer by matching *outputs* on that layer's inputs, $W = (\sum_i G_i)^{-1} \sum_i G_i W_i$; in input directions with negligible activation variance the solution is essentially unconstrained, so W acquires large components that are functionally inert on in-distribution inputs. The resulting merge point is over 450 from the base model in parameter norm—larger than the encoder norm itself (336)—while scoring a healthy 0.723, which is only possible because most of that displacement lies in inference-invisible directions. Every ingredient of APR reads the anchor $v_i = \theta_i - \theta$, so from this initialization v_i inherits that inflated, non-functional component: the step $-g \odot |v|$ scales noise on directions where $|v|$ is huge but the gradient is near zero, and the trust region $\gamma|v|$ is widest exactly there—which is also why the usable rate collapses by an order of magnitude here. This yields a concrete prediction: anchoring on the functional part of v (equivalently, refining RegMean from a task-vector-defined anchor) should remove the handicap—an experiment we have not yet run. It also implies that parameter-space

distance from base is a valid drift proxy for the task-vector-based methods but not for RegMean, whose functional drift must be measured in output space.

When sixty-four examples do not overfit—and when eight do. At the full budget the strongest labeled-replay result is also the most surprising: from the RegMean initialization, ordinary GD at its safe rate can be run to *near-interpolation of the replay buffer*—over $S = 60$ sweeps (1,200 full-batch passes over the same 64 examples per task) the replay loss falls to under 3% of its starting value—while test accuracy *rises* to 0.768 and worst-task holds at 0.51. Three structural facts explain the absence of memorization there: the zero-shot head is frozen, so the cheap channel for fitting arbitrary labels is unavailable (unfreezing it on the same 64 examples is catastrophic, Table 9); the total parameter displacement is minute ($\approx$0.05% of the encoder norm), bounding the capacity of the perturbation; and twenty interleaved tasks overwrite each other's sample-specific noise directions while preserving the shared task-restoration signal. Consistent with this account, the class-balanced buffers of the 397-class SUN397 task cover at most 64 of its classes, yet all-class test accuracy improves—transfer through shared features by construction.

The budget grid shows where this benignity ends, and a methodological point matters here: best-per-cell tables cannot detect overfitting, because test-selected configurations exclude it by construction, so we audit *every* (η, S, n) cell for the overfitting signature (replay loss improving from $S = 30$ to $S = 60$ while test accuracy falls). For unanchored GD the signature appears as soon as the buffer is small: 9 of 36 small-n cells deteriorate, with textbook cases at $n = 16$ (replay $0.083 \to 0.063$ while test falls $0.636 \to 0.288$; a second cell falls $0.614 \to 0.540$), plus two outright instabilities where replay worsens as well. A practical corollary is that GD's safe learning-rate window narrows as sweeps accumulate—a rate tuned at $S = 5$ cannot be reused at $S = 60$. For the anchored update the signature never appears: across $\approx$48 cells the largest decline is 0.003, with no instabilities, *even though* APR also reaches near-interpolation (replay 0.007–0.011 on $n = 8$ buffers) and its optimal rate transfers unchanged across horizons. The absence of overfitting is therefore a property of the trust region, not of the data regime: once every coordinate is capped at $\gamma|v_{i,r}|$, additional sweeps have nowhere damaging to go, whereas GD keeps moving after it has fit the buffer.

Geometry of the twenty-task regime. Direct measurement explains why small careful steps are competitive here. At the tuned merge, every expert sits at distance $\approx$2.4 (encoder norm 336), but the best refined solutions found by *any* stable method lie only 0.14–0.23 from a strong initialization—about 8% of one expert distance—whereas APR's matched-budget optimum travels 0.5–1.2. As tasks accumulate, the achievable joint optimum stays close to a good initialization (a compromise among twenty mutually incompatible experts, mirrored by the tuned λ falling from 0.2–0.3 to 0.08), while the expert distances $|v_i|$ that set APR's stride stay large; the two length scales that coincided at three and eight tasks decouple at twenty. This is the mechanism behind both the shrinking gate advantage and the shrinking matched-budget margin over GD, and it makes a testable prediction, which we then confirm below: from the *base model*, where the destination genuinely is far, the distance-scaled step should dominate small-step GD again.

Refining directly from the base model. Setting $\lambda = 0$ makes the "merge" the pretrained encoder itself, so refinement runs from θ_0 with no merge at all—both a "do you need to merge?" ablation and a direct test of the geometry above. The prediction holds sharply: from the base model APR beats ordinary GD by +0.058 mean accuracy at matched budget and by the same margin at $S = 30$ (0.683 vs. 0.625), roughly double its from-merge margin, because the good region is genuinely far (> 1) while GD's small steps travel only 0.09–0.22 and cannot reach it. More striking

Table 13: Parameter-space distance $\|\theta - \theta_0\|$ from the base encoder (norm 336; every expert lies ≈2.4 from base), measured exactly for the task-vector-based merges and for refinement started from θ_0. Refining from the base model reaches merge-level accuracy while staying several times closer to the pretrained weights than any merge. RegMean is omitted: its distance (> 450) is dominated by inference-invisible directions and does not reflect functional drift (see "Composition, again").

Model point	Dist. from θ_0	Mean acc.
Task-arithmetic merge (λ=0.08)	1.04	0.604
AdaMerging-layer merge	1.68	0.665
TIES merge	2.95	0.626
APR from base, S=5	**0.29**	0.644
APR from base, S=30	0.59	0.683

is the proximity result (Table 13): APR from the base model reaches 0.644 at $S = 5$—already *above* the tuned task-arithmetic merge (0.604)—while ending only 0.29 from θ_0, against the merge's 1.04; pushing to $S = 30$ reaches 0.683 at displacement 0.59, still below every merge's distance from base and above every checkpoint-space merge's accuracy. Refinement-from-base thus traces an accuracy–proximity frontier that dominates the standard merge on both axes: comparable or better multitask accuracy while straying several times less far from the pretrained model. Since proximity to base suggests preserved zero-shot behavior on *unseen* tasks, this is a promising minimal-drift operating point; the next paragraph replaces that suggestion with a direct measurement.

Held-out-task retention: the minimal-drift point, measured. We split the suite into 14 training tasks (the specialized, domain-shifted ones) and 6 held-out tasks—the natural-image tasks on which the base model's zero-shot accuracy is highest (SUN397, STL10, CIFAR-10, Pets, Food101, Flowers102; base mean 0.802, so there is something to lose). Every merge and every refinement step sees only the 14 training tasks; the held-out tasks are evaluated purely zero-shot through their frozen text-derived heads, so nothing task-specific leaks. Table 14 and Figure 1 report training-task accuracy against held-out retention for the full baseline tier (single seed). The result is unambiguous: **the accuracy–retention Pareto frontier is populated almost entirely by the refinement**. Its high-accuracy end is APR from AdaMerging (0.682 train at −0.022 retention drop—+0.146 training accuracy over the best merge at *equal* retention, and better than the AdaMerging merge alone on *both* axes), followed by APR from the base model (0.637/ − 0.022 and 0.618/ − 0.012); every checkpoint-only and label-free merge, and the refinement from the task-arithmetic merge, is dominated. Three controls sharpen the claim. Ordinary descent from the base model matches APR's retention only by barely moving: at the learning rate that reaches comparable retention (−0.012), it delivers 0.069 *less* training accuracy than APR, so the frontier position is a property of the anchored refinement, not merely of starting at θ_0. The ungated anchored update at matched displacement loses on both axes (−0.061 train, −0.017 retention)—the first setting in which the gate is clearly not neutral, plausibly because restricting updates to coordinates that help a training task avoids collateral damage to shared features, though with a single ungated learning rate run this is preliminary. And RegMean provides an independent confirmation of the null-space account given earlier: its merge sits 386 from the base model in parameter norm—farther than $\|\theta_0\|$ itself—yet loses only 0.059 of held-out accuracy, while task arithmetic at $\lambda = 0.2$, at distance 2.2, loses 0.208. Parameter distance therefore fails as a drift metric *across* families (Breadcrumbs at 1.28 retains best of all merges; within the task-arithmetic family the relationship is monotone), which is why retention must be measured rather than proxied—and it supplies a practical warning: tuning the

Table 14: Held-out-task retention on the twenty-task suite: merge/refine on 14 specialized tasks, evaluate the 6 strongest natural-image tasks zero-shot (base mean 0.802); single seed, $n = 64$, refinements at S=30 cosine. * marks the training-accuracy/retention Pareto frontier. Every merge is dominated by a refinement point; RegMean's parameter distance is functionally inert (see text).

Model (built from the 14 train tasks)	Dist. θ_0	Train-14	Held-out 6 (drop)
Base model θ_0*	0.00	0.443	0.802 (0.000)
GD from base, η=5e−5*	0.08	0.529	0.797 (−0.005)
GD from base, η=1e−4	0.12	0.549	0.790 (−0.012)
APR from base, η=4*	0.44	0.618	0.790 (−0.012)
APR from base, η=8*	0.60	0.637	0.780 (−0.022)
Ungated from base, η=2	0.61	0.576	0.763 (−0.039)
Task arithmetic (λ=0.08)	0.87	0.536	0.781 (−0.021)
APR from task arith., η=4	1.06	0.654	0.772 (−0.030)
Breadcrumbs*	1.28	0.536	0.792 (−0.010)
APR from AdaMerging, η=4*	1.44	**0.682**	0.780 (−0.022)
AdaMerging-layer	1.69	0.614	0.776 (−0.026)
Task arithmetic (λ=0.2)	2.18	0.510	0.594 (−0.208)
TIES (λ=0.6)	2.90	0.578	0.770 (−0.032)
DARE-TIES	3.31	0.573	0.753 (−0.049)
RegMean	386	0.675	0.743 (−0.059)

merge coefficient on the merged tasks alone is actively harmful, since $\lambda = 0.08 \to 0.2$ loses accuracy on both axes (-0.026 train, -0.187 held-out).

Replay budget at twenty tasks. The eight-task budget study (Table 5) is extended to the twenty-task suite with a full grid: $n \in \{8, 16, 32\}$ labeled examples per task, each refined at $S \in \{5, 30, 60\}$ sweeps with the learning rate tuned per cell, from all four starting points, with identical buffer draws across runs ($n = 64$ from the runs above; single seed). Table 15 reports the best configuration over the learning rate *and* the horizon, since the two interact: at small n nearly every best cell uses $S \geq 30$—more passes genuinely compensate for fewer examples—and the optimal rate falls as the buffer shrinks (to $\eta \approx 1$–4), while extending further to $S = 60$ moves best configurations by at most 0.004 at any budget, so the horizon is effectively converged by thirty sweeps. Three regularities. First, degradation with budget is graceful in the mean but concentrated in the *worst task*: from AdaMerging, mean accuracy loses only 0.03 across the 8× budget reduction ($0.712 \to 0.683$) while worst-task accuracy loses 0.14 ($0.312 \to 0.173$)—the honest cost of a small buffer is paid almost entirely by the hardest task, which a twenty-task mean conceals. Second, the refinement's value survives at the smallest budget: with just *eight* labeled examples per task, refining the base model reaches 0.632 (0.626 ± 0.005 over five buffer draws, above the merge on every draw)—while the tuned task-arithmetic merge (0.604) uses no labels but requires all twenty expert checkpoints and a tuned coefficient. Third, from the base model the anchored update's margin over plain gradient descent is large and essentially independent of the budget ($\approx +0.05$–0.06 mean at every n; GD row in Table 15): the geometry argument above, not data volume, sets that gap.

Selecting hyperparameters without the evaluation set. All twenty-task numbers above select each configuration on the evaluation split, which is optimistic in absolute terms. A pilot of the strict protocol splits the historical 64 labeled examples per task disjointly into 32 for the refinement sweeps and 32 held out for selection, and picks the configuration by held-out accuracy. From the

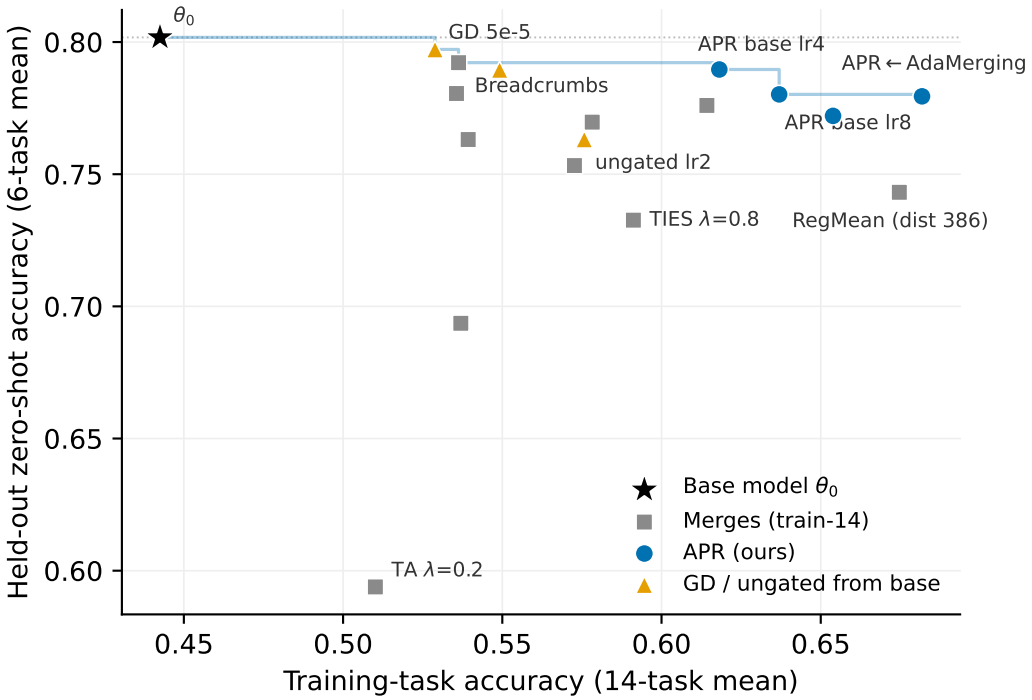

Figure 1: The accuracy–retention trade-off of Table 14: each point is a model built from the 14 training tasks, evaluated on them (x) and zero-shot on the 6 held-out tasks (y; dotted line = the base model's own zero-shot accuracy). The step line is the Pareto frontier: its high-accuracy half is populated entirely by the anchored refinement, while every merge (gray squares) is dominated. RegMean sits at parameter distance 386 from θ_0 yet retains mid-pack—parameter distance is not a drift metric (see text).

RegMean initialization the selection gap is negligible—the validation-selected configuration loses 0.001 of test accuracy against the evaluation-oracle choice—so the evaluation-selected tables above are not materially inflated. Halving the sweep budget from 64 to 32 examples per task costs ≈0.02 mean accuracy, consistent with the budget curve of Table 15. The held-out buffer is thus a practical selection instrument at essentially no cost in final quality; the multi-seed replication below therefore pins the seed-0 configurations rather than re-selecting per seed.

Multi-seed replication of the headline cells. Every twenty-task number above is a single draw of the replay buffers. We replicate the headline cells over five buffer seeds, holding the selected configurations fixed (justified by the negligible selection gap just measured), so that the variance isolates the buffer draw: the TIES and AdaMerging initializations are identical across seeds, while the RegMean initialization legitimately redraws with its Gram inputs. Table 16 reports the outcome; the single-seed conclusions survive, and two sharpen. First, the gate's mean-accuracy neutrality is confirmed with tight intervals (paired APR-minus-ungated: -0.002 ± 0.006 from TIES, $+0.001 \pm 0.005$ from AdaMerging), but the *worst-task* effect is initialization-dependent: from AdaMerging the gate helps on all five seeds ($+0.031 \pm 0.015$), while from TIES it is mildly negative—so "neutral" is the right summary for the mean, not uniformly for the fragile tasks. Second, against the plain-descent probe the paired margins are consistent across seeds where the single seed had suggested a tie: $+0.007 \pm 0.004$ mean from TIES, $+0.018 \pm 0.007$ from AdaMerging (with a worst-task margin of

Table 15: Replay-budget study on the twenty-task suite: best configuration over learning rate and horizon $S \in \{5, 30, 60\}$ per cell, mean top-1 accuracy (worst-task in parentheses); single seed, identical buffer draws across cells at each n. "Init. alone" is budget-independent. Ordinary GD is reported from the base model (the from-base ablation's comparison); at small budgets its gap to APR is unchanged from $n = 64$. Worst-task accuracy degrades $\approx 3\times$ faster than the mean as the buffer shrinks.

Initialization	Method	n=8	n=16	n=32	n=64
Base model (0.550 (0.100))	+ GD	0.573 (0.093)	0.580 (0.113)	0.602 (0.160)	0.625 (0.207)
	+ APR	0.632 (0.187)	0.640 (0.205)	0.654 (0.236)	0.683 (0.350)
Task arith. (0.604 (0.115))	+ APR	0.650 (0.197)	0.659 (0.242)	0.674 (0.274)	0.693 (0.351)
TIES (0.626 (0.150))	+ APR	0.665 (0.224)	0.673 (0.250)	0.691 (0.284)	0.708 (0.386)
AdaMerging (0.665 (0.101))	+ APR	0.683 (0.207)	0.688 (0.249)	0.698 (0.241)	0.712 (0.312)

Table 16: Multi-seed replication on the twenty-task suite: five replay-buffer seeds, configurations pinned at the seed-0 selections (mean ± std of mean / worst-task top-1 accuracy). Initializations are seed-invariant except RegMean, whose Gram inputs redraw with the buffer. The ungated anchored update is shown where the gate comparison is the point; the base-model rows retain the plain-descent reference. GD's n=64 from-base row is dominated by one diverged seed (see text).

Cell (5 seeds)	Method	Mean acc.	Worst-task
TIES, S=5	APR	0.697 ± 0.006	0.261 ± 0.024
	ungated	0.699 ± 0.003	0.276 ± 0.019
AdaMerging, S=5	APR	0.703 ± 0.003	0.177 ± 0.008
	ungated	0.702 ± 0.004	0.146 ± 0.008
TIES, S=30	APR	0.708 ± 0.005	0.369 ± 0.013
AdaMerging, S=30	APR	0.712 ± 0.003	0.321 ± 0.016
RegMean, S=30	APR	0.764 ± 0.002	0.501 ± 0.011
Base, n=64	APR	0.686 ± 0.004	0.340 ± 0.017
	GD	0.523 ± 0.187	0.142 ± 0.072
Base, n=8	APR	0.626 ± 0.005	0.170 ± 0.017
	GD	0.572 ± 0.003	0.095 ± 0.011

$+0.162 \pm 0.018$), $+0.053 \pm 0.006$ from the base model at $n = 8$, and $+0.199 \pm 0.060$ worst-task at $n = 64$. The two data-anchored results also replicate: refining the base model on *eight* labels per task gives 0.626 ± 0.005, above the tuned task-arithmetic merge on every seed, and the RegMean initialization remains the one place plain descent finishes ahead—by a small but seed-consistent 0.003 ± 0.002, in line with the corrupted-anchor account above.

Summary of the twenty-task study. Five claims survive the full campaign, each with its evidence. (i) *The anchored refinement improves every initialization it is given*—six merges spanning the checkpoint-only, label-free, and data-fitted tiers, and the base model itself—with the gain shrinking as the initialization improves but never vanishing (Tables 11, 12); the headline cells carry five-seed error bars of 0.002–0.006 (Table 16). (ii) *The load-bearing ingredient is the anchored, distance-scaled, clipped step, not the attribution gate*: the gate is neutral on mean accuracy (five-seed paired differences within ±0.006), its thresholded variants cannot help because the gradient's coordinate-level selection carries no transferable structure (the held-out audit), and its residual value

is initialization-dependent worst-task repair—plus, preliminarily, held-out retention. (iii) *Safety is the method's clearest practical advantage*: across every learning rate, horizon, budget, and buffer draw tested, no anchored-refinement run ever fell below its initialization or the zero-shot floor, while the unconstrained-descent probe has catastrophic cells at adjacent learning rates, overfits small buffers at long horizons, and can diverge under nothing more than a replay-buffer redraw. (iv) *The method works in the small-data regime*: eight labeled examples per task suffice to beat the tuned task-arithmetic merge from the base model alone, with the budget's cost concentrated in worst-task accuracy (Table 15). (v) *Refinement offers an accuracy–retention frontier that merging does not*: on tasks never merged or refined, the frontier's high-accuracy half is populated entirely by the refinement, every merge is dominated, and the best cell gains 0.146 training accuracy over the best merge at equal held-out retention (Figure 1).

What is still missing. The baseline tiers are now populated (checkpoint-only, label-free with RegMean and AdaMerging, and the matched labeled tier including learned Localize-and-Stitch), and the headline configurations carry error bars. The remaining gaps are protocol-level: the hyperparameter *sweeps* behind Tables 8, 3, and the twenty-task tables of Section 4.3 are still single-seed, and are evaluation-selected except for the disjoint-holdout pilot reported there (which found the selection gap negligible); the headline twenty-task cells now carry five-seed error bars (Table 16) but the full grids behind Tables 12 and 15 remain single-seed; the predicted functional-anchor fix for the RegMean initialization is not yet run; the held-out retention study (Table 14) is single-seed, and its gate-helps-retention observation rests on a single ungated learning rate; the shared-output track of Section 4.2 is single-seed with refinement grids whose optima are not yet bracketed; and the decoder-LLM setting promised in the experimental protocol remains the subject of ongoing work. The matched-budget comparison already answers the central alternative explanation—that a small labeled sample would help *any* reasonable method equally: at 64 labeled examples per task, most alternative uses of the same data are markedly worse than the refinement, and the strongest competitors are themselves parameter-space edits of the merge.

5 Expected Contribution

The expected contribution is an attribution-patching perspective on replay-guided model merging. Rather than using attribution scores to build a pre-merge sparsification mask, the method starts from a standard task-arithmetic merge and asks a local intervention question: for task i, which coordinates would reduce the replay loss if the current model moved toward expert i? The answer is the sign of the coordinate-wise product $g_{i,r}v_{i,r}$, which coincides with a trust-region acceptance rule that keeps each step moving toward its expert, so the attribution-patching and replay-refinement views motivate the same procedure. This leads to a simple implementable algorithm: mask out expert directions that are locally loss-increasing, scale the remaining negative-gradient update by distance to the expert, clip each per-task update by a trust-region fraction, and immediately apply that task's update before computing the next task's gradient and gate. Empirically, the refinement is best understood as a *complement* to merging rather than a replacement for it: it improves every merge initialization we have tried—task arithmetic, TIES, DARE-TIES, Breadcrumbs, AdaMerging, and RegMean—and the strongest results always come from composing with the strongest available merge, never from plain task arithmetic. Where the thresholded gate adds nothing over the ungated anchored step (GLUE encoders, and the twenty-task CLIP suite), a longer refinement with randomized task order and an annealed step size is currently the more robust choice; the anchored trust-region step itself, however, retains its advantage and its hyperparameter safety at

every scale tested. Two further findings sharpen the picture at scale: on the twenty-task suite the refinement remains useful but its margin shrinks as the initialization improves, and running it directly from the *pretrained* model reaches merge-level accuracy while staying several times closer to the base weights—a minimal-drift operating point that merging does not offer, and one whose promise is confirmed by direct measurement: on tasks never merged or refined, the refinement populates the accuracy–retention Pareto frontier while every merge in the comparison is dominated. On the shared-output T5 track (Section 4.2), which drops the oracle-task-identity assumption entirely, the same composition picture recurs. The proposal also contributes a stricter empirical protocol: fair labeled-replay baselines, an aggregated-update ablation, mask-stability tests, systematic GLUE pair/subset evaluations, and secondary shared-head or CLIP/ViT benchmarks. If successful, the method would provide a lightweight bridge between checkpoint-only task arithmetic and full multitask fine-tuning with replay buffers, while making clear which gains come from the attribution gate rather than from supervised replay alone.

References

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, 2019.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.

Michael S. Matena and Colin A. Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022.

Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *International Conference on Learning Representations*, 2023.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.

Mohammad Davari and Eugene Belilovsky. Model Breadcrumbs: Scaling multi-task model merging with sparse masks. *arXiv preprint arXiv:2312.06795*, 2023.

Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2024.

Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-Merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.

Fanshuang Kong, Richong Zhang, and Ziqiao Wang. Activated parameter locating via causal intervention for model merging. *arXiv preprint arXiv:2408.09485*, 2024.

Shwai He, Runqi Wang, Jindong Wang, Jindong Gu, and Xing Xie. Localize-and-Stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.

Zhenyi Lu, Chengxu Yin, Wujie Wang, and Xuebo Liu. Twin-Merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.

Anke Tang, Li Shen, Yong Luo, Liang Ding, Han Hu, Bo Du, and Dacheng Tao. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.

Zihan Zeng, Jiahui Chen, Lei Li, and Min Zhang. Efficient model editing with task vector bases: A theoretical framework and scalable approach. *arXiv preprint arXiv:2502.01015*, 2025.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Xingrun Xing. LM-Cocktail: Resilient tuning of language models via model merging. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jiménez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. In *International Conference on Machine Learning*, 2024.

Yifei He, Siqi Zeng, Yuzheng Hu, Rui Yang, Tong Zhang, and Han Zhao. MergeBench: A benchmark for merging domain-specialized LLMs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning*, 2017.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2024.

Janos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing language model behavior to components. *arXiv preprint arXiv:2403.00745*, 2024.

A Box-Distance Monotonicity and the Expert Box

This appendix proves the general geometric result behind Corollary 1. The statement is self-contained: $x_1, \dots, x_n$ are arbitrary points in $\mathbb{R}^D$ (in the application, the task experts $\theta_1, \dots, \theta_T$), and the superscript d indexes coordinates.

Theorem 1 (Box-distance monotonicity under coordinatewise movement toward a hull point)**.** *Let $x_1, \dots, x_n \in \mathbb{R}^D$. Define the axis-aligned box hull*

$$B := \prod_{d=1}^{D} [m_d, M_d], \qquad m_d := \min_{1 \le j \le n} x_j^d, \qquad M_d := \max_{1 \le j \le n} x_j^d.$$

Let $y, z \in \mathbb{R}^D$. Suppose that there exists some $i \in \{1, \dots, n\}$ such that, for every coordinate $d \in \{1, \dots, D\}$, the point z^d lies between x_i^d and y^d. Equivalently,

$$|z^d - x_i^d| \le |y^d - x_i^d|$$

and $z^d - x_i^d$ has the same sign as $y^d - x_i^d$, allowing equality.

Then, for every $1 \le p \le \infty$,

$$d_p(z, B) \le d_p(y, B),$$

where $d_p(u, B)$ denotes the distance from u to B with respect to the ℓ^p-norm.

Proof. For each coordinate d, set

$$I_d := [m_d, M_d].$$

Since $x_i \in B$, we have

$$x_i^d \in I_d$$

for every d.

Define the one-dimensional distance function

$$\delta_d(t) := \operatorname{dist}(t, I_d) = \inf_{s \in I_d} |t - s|.$$

We first prove the following elementary one-dimensional claim.

Claim. Let $I = [m, M] \subset \mathbb{R}$, let $c \in I$, and suppose that z lies between c and y. Then

$$\operatorname{dist}(z, I) \le \operatorname{dist}(y, I).$$

Indeed, if $y \in I$, then since $c \in I$ and I is an interval, every point between c and y also lies in I. Hence $z \in I$, so

$$\operatorname{dist}(z, I) = 0 = \operatorname{dist}(y, I).$$

If $y > M$, then

$$\operatorname{dist}(y, I) = y - M.$$

Since z lies between $c \in I$ and $y > M$, we have $z \le y$. If $z \le M$, then $\operatorname{dist}(z, I) = 0$. If $z > M$, then

$$\operatorname{dist}(z, I) = z - M \le y - M = \operatorname{dist}(y, I).$$

The case $y < m$ is symmetric. This proves the claim.

Now apply the claim coordinatewise with

$$I = I_d, \qquad c = x_i^d.$$

Because z^d lies between x_i^d and y^d, we obtain

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every $d \in \{1, \dots, D\}$.

Next, distance to an axis-aligned box separates coordinatewise. Namely, for $1 \le p < \infty$,

$$d_p(u, B) = \left(\sum_{d=1}^{D} \delta_d(u^d)^p \right)^{1/p},$$

and for $p = \infty$,

$$d_\infty(u, B) = \max_{1 \le d \le D} \delta_d(u^d).$$

This follows because the closest point of B to u is obtained by clipping each coordinate u^d to the interval $I_d = [m_d, M_d]$.

Since

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every d, monotonicity of the ℓ^p-norm on the nonnegative orthant gives

$$d_p(z, B) \le d_p(y, B).$$

This proves the theorem. □

The box hull B may look like a loose relaxation of the ordinary convex hull of the experts, but it is canonical in its own right: it is the smallest set containing the experts that is convex with respect to ℓ^1-geodesics.

Proposition 1. *The set B is the ℓ^1-geodesic convex hull of $\{x_1, \dots, x_n\}$.*

Proof. For $a, b \in \mathbb{R}^D$, define the ℓ^1-metric interval between a and b by

$$I_1(a, b) := \{u \in \mathbb{R}^D : \|a - u\|_1 + \|u - b\|_1 = \|a - b\|_1\}.$$

We claim that

$$I_1(a, b) = \prod_{d=1}^{D} [\min(a^d, b^d), \max(a^d, b^d)].$$

Indeed, by the triangle inequality in one dimension,

$$|a^d - u^d| + |u^d - b^d| \ge |a^d - b^d|$$

for each coordinate d. Summing over d, equality in the ℓ^1-triangle inequality holds if and only if equality holds in every coordinate. In one dimension, equality holds precisely when u^d lies between a^d and b^d. Hence the displayed formula for $I_1(a, b)$ follows.

Now B is clearly ℓ^1-geodesically convex: if $a, b \in B$, then every point coordinatewise between a and b also lies in B.

It remains to show minimality. Let $C \subseteq \mathbb{R}^D$ be any ℓ^1-geodesically convex set containing $x_1, \ldots, x_n$. Since C is closed under ℓ^1-metric intervals, it is closed under taking coordinatewise boxes between pairs of its points. In particular, if $a, b \in C$, then both the coordinatewise minimum

$$a \wedge b := (\min(a^1, b^1), \ldots, \min(a^D, b^D))$$

and the coordinatewise maximum

$$a \vee b := (\max(a^1, b^1), \ldots, \max(a^D, b^D))$$

belong to C.

By induction, C therefore contains the coordinatewise minimum

$$m := (m_1, \ldots, m_D)$$

and the coordinatewise maximum

$$M := (M_1, \ldots, M_D)$$

of the points $x_1, \ldots, x_n$. Since C is ℓ^1-geodesically convex, it contains the entire ℓ^1-metric interval between m and M. But

$$I_1(m, M) = \prod_{d=1}^{D} [m_d, M_d] = B.$$

Therefore every ℓ^1-geodesically convex set containing $x_1, \ldots, x_n$ also contains B. Hence B is the smallest such set, i.e.

$$B = \mathrm{gconv}_{\ell^1}\{x_1, \ldots, x_n\}.$$

□